

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Гневшев А. Ю. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:
**РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И
МОНИТОРИНГА ИМИТАЦИОННЫХ СЕРВИСОВ В СРЕДЕ
НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль)
«Системная и программная инженерия»

Студент: _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

Москва 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль)
«Системная и программная инженерия»

Тема ВКР	Разработка системы для проксирования и мониторинга имитационных сервисов в среде нагрузочного тестирования
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Автоматизированная информационная система для централизованного мониторинга и управления распределённой инфраструктурой имитационных сервисов (заглушек) в среде нагрузочного тестирования.
Основные функции	1. Управление жизненным циклом имитационных сервисов: регистрация, запуск, остановка и удаление процессов на локальных и удалённых хостах без установки агентов. 2. Динамическое проксирование HTTP-запросов к целевым имитационным сервисам с сохранением метода, заголовков и тела запроса. 3. Ограничение интенсивности запросов (Rate Limiting) на уровне каждого имитационного сервиса с изменением параметров в реальном времени без перезапуска. 4. Мониторинг состояния инфраструктуры и экспорт метрик в формате Prometheus. 5. Обеспечение сохранности конфигураций и автоматическое восстановление активных сервисов после перезапуска системы. 6. Визуализация данных и управление системой через веб-интерфейс, позволяющий производить настройку хостов и просмотр журналов процессов.
Используемые технологии и платформы	Python 3.10+, FastAPI, Uvicorn, Redis (AOF), asyncssh, httpx, Jinja2, cryptography/Fernet, Prometheus, OpenJDK 17+, ОС Astra Linux Special Edition.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Провести анализ предметной области и существующих подходов к управлению имитационными средами в процессах обеспечения качества ПО. 2. Спроектировать архитектуру программного комплекса, обеспечивающую гибкое масштабирование и работу в различных режимах. 3. Обосновать выбор технологического стека и программных средств реализации системы. 4. Разработать алгоритмы динамического проксирования трафика, механизмы контроля интенсивности запросов и программные интерфейсы для автоматизации управления. 5. Реализовать систему хранения конфигураций и контроля состояний исполняемых файлов. 6. Провести тестирование разработанного комплекса в среде ОС Astra Linux и оценить эффективность реализованных механизмов мониторинга и управления.
Состав технической документации	1. Пояснительная записка.
Состав графической части	1. Диаграмма системы в нотации С4 – 3 экз. 2. Диаграмма развёртывания – 1 экз. 3. ER-диаграмма объектов в Redis – 1 экз. 4. Макеты пользовательского интерфейса – 6 экз. 5. Экраны реализованного интерфейса – 5 экз. 6. Графики результатов тестирования – 3 экз. 7. Презентация – 1 экз.

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Согласование темы с научным руководителем																		
Анализ предметной области и существующих решений																		
Обоснование технологического стека и формирование требований																		
Проектирование архитектуры системы																		
Разработка модулей управления процессами и проксирования																		
Реализация веб-интерфейса и модуля мониторинга																		
Тестирование и развёртывание в среде Astra Linux																		
Оформление пояснительной записки ВКР																		
Нормоконтроль, подготовка к защите																		

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Гневшев Александр Юрьевич /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа на тему: «Разработка системы для проксирования и мониторинга имитационных сервисов в среде нагрузочного тестирования».

В первой главе проведён анализ предметной области нагрузочного тестирования и роли имитационных сервисов, выполнен обзор и сравнение существующих аналогов, обоснован выбор технологического стека: Python, FastAPI, Redis. Во второй главе описано проектирование архитектуры системы с применением нотации C4 Model, а также проектирование и реализация модулей динамического HTTP-проксирования, ограничения частоты запросов (Rate Limiting) и веб-интерфейса управления. В третьей главе рассмотрены процедура развёртывания комплекса в ОС Astra Linux, методика и сценарии функционального и нагрузочного тестирования, а также вопросы эксплуатации системы. Библиографический список состоит из 52 источников, из них 26 электронных.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	11
1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов	11
1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения	12
1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека	15
1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса	20
1.5 Формирование функциональных и технических требований к системе	27
1.6 Выводы	30
2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ	31
2.1 Разработка архитектуры распределённого программного комплекса	31
2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis	36
2.3 Разработка макетов приложения	41
2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.	43
2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting	45
2.6 Проектирование и программная реализация пользовательского интерфейса	46
2.7 Реализация механизма экспорта метрик мониторинга	51
2.8 Выводы	53

3	ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА	56
3.1	Описание процесса развёртывания системы в среде операционной системы Astra Linux	56
3.2	Тестирование механизмов автоматического восстановления состояний и перезапуска сервисов	59
3.3	Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой	65
3.4	Выводы	71
	ЗАКЛЮЧЕНИЕ	73
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	75
	ПРИЛОЖЕНИЕ А	80
	ПРИЛОЖЕНИЕ Б	94
	ПРИЛОЖЕНИЕ В	97

ВВЕДЕНИЕ

Актуальность темы. В современной практике обеспечения качества сложных программных комплексов проведение нагрузочного тестирования требует создания стабильной и управляемой среды [1,2]. Одной из ключевых проблем при организации такой среды является управление инфраструктурой имитационных сервисов (заглушек) – специализированного программного обеспечения, воспроизводящего поведение реальных внешних компонентов системы. Применение имитационных сервисов позволяет изолировать тестируемую систему от реальных внешних зависимостей, обеспечивая воспроизводимость тестовых сценариев и возможность моделирования различных режимов работы, в том числе нештатных. В условиях распределённых архитектур количество задействованных заглушек существенно возрастает, а их ручное сопровождение становится трудоёмким, что обуславливает необходимость автоматизированных средств управления.

В условиях реализации политики импортозамещения в Российской Федерации актуальность приобретает разработка отечественных инструментов управления инфраструктурой, способных заменить зарубежные программные решения и обеспечить совместимость с сертифицированными отечественными операционными системами [3].

Цель работы – проектирование и разработка автоматизированной информационной системы для централизованного управления имитационными сервисами в среде нагрузочного тестирования и мониторинга их состояния.

Задачи исследования:

1. Провести анализ предметной области и существующих подходов к управлению имитационными средами в процессах обеспечения качества ПО.
2. Спроектировать архитектуру программного комплекса, обеспечивающую гибкое масштабирование и работу в различных режимах.
3. Обосновать выбор технологического стека и программных средств реализации системы.

4. Разработать алгоритмы динамического проксирования трафика, механизмы контроля интенсивности запросов и программные интерфейсы для автоматизации управления.

5. Реализовать систему хранения конфигураций и контроля состояний исполняемых файлов.

6. Провести тестирование разработанного комплекса в среде ОС Astra Linux и оценить эффективность реализованных механизмов мониторинга и управления.

Объект исследования – инфраструктура запуска и эксплуатации имитационных сервисов (заглушек) в среде нагрузочного тестирования.

Предмет исследования – механизмы и алгоритмы автоматизированного управления и контроля состояния имитационных сервисов в среде нагрузочного тестирования.

Научная новизна работы заключается в разработке алгоритма централизованного управления распределённой средой, поддерживающего индивидуальную настройку ограничений интенсивности запросов (Rate Limiting) для каждого имитационного сервиса в отдельности, а также в реализации агентно-независимой схемы взаимодействия с удалёнными узлами посредством протоколов SSH/SCP (Secure Shell, Secure Copy Protocol) без установки дополнительного программного обеспечения на целевые серверы.

Практическая значимость работы состоит в создании программного инструмента, позволяющего отказаться от использования зарубежных систем управления, упростить эксплуатацию инфраструктуры имитационных сервисов и обеспечить её интеграцию со стандартными средствами мониторинга (Prometheus).

Положения, выносимые на защиту:

1. Архитектура программного комплекса, поддерживающая гибкое масштабирование системы от локального запуска на одном хосте до формирования распределённого кластера.

2. Механизм динамического проксирования запросов, реализующий прозрачную маршрутизацию входящего трафика к целевым имитационным сервисам.

3. Алгоритм контроля интенсивности запросов, обеспечивающий стабильность работы имитационной среды при пиковых нагрузках.

4. Методика формирования и экспорта метрик производительности для непрерывного мониторинга состояния инфраструктуры загрузок.

Структура работы. Выпускная квалификационная работа включает введение, три главы основного текста, заключение, список использованных источников и приложения [4].

Во введении обоснована актуальность исследования, поставлены цель и задачи, определены объект и предмет работы, указаны научная новизна и практическая значимость полученных результатов.

В первой главе проводится анализ предметной области, исследуются существующие аналоги и методы управления имитационными средами. На основе сравнительного анализа обосновывается выбор технологий и формируются требования к системе.

Во второй главе описывается проектирование архитектуры системы, структуры хранения данных и логики работы основных модулей. Приводятся схемы взаимодействия компонентов и алгоритмы реализации функций проксирования и управления кластером.

В третьей главе рассматриваются вопросы практической реализации, настройки и эксплуатации системы в среде ОС Astra Linux. Приводятся результаты тестирования, подтверждающие корректность работы разработанных механизмов управления и мониторинга.

В заключении сформулированы основные выводы, подтверждающие решение поставленных задач и достижение цели работы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов

В современной практике обеспечения качества сложных программных комплексов нагрузочное тестирование занимает критически важное место [1,2]. Оно позволяет гарантировать стабильность и управляемость информационной системы при высоких эксплуатационных нагрузках. Одной из ключевых проблем при организации таких испытаний является необходимость взаимодействия объекта тестирования с множеством внешних систем и сервисов.

Использование реальных внешних интеграций в среде тестирования часто оказывается невозможным или нецелесообразным по ряду причин:

1. Ограниченность ресурсов: внешние системы могут иметь жёсткие лимиты на количество запросов или требовать значительных финансовых затрат на эксплуатацию.

2. Изоляция среды: для получения достоверных результатов необходимо исключить влияние задержек и сбоев сторонних систем на показатели исследуемого объекта.

3. Риски изменения данных: проведение тестов на реальных интеграциях может привести к нежелательному изменению или повреждению информации в продуктовых базах данных.

Для решения этих проблем применяются имитационные сервисы («заглушки») – специализированное программное обеспечение, которое воспроизводит поведение реальных компонентов, отвечая на запросы тестируемой системы заранее определённым образом [5]. В контексте данной работы в качестве имитационных сервисов рассматриваются Java-приложения в форматах .jar и .war.

Роль имитационных сервисов в инфраструктуре нагрузочного тестирования заключается в следующем:

1. Обеспечение автономности стенда: заглушки позволяют полностью изолировать тестируемую систему от внешнего окружения, создавая предсказуемую среду.

2. Эмуляция различных сценариев: имитационные сервисы могут воспроизводить не только корректные ответы, но и ошибки, что необходимо для проверки механизмов отказоустойчивости.

3. Контроль интенсивности трафика: использование механизмов ограничения запросов (Rate Limiting) позволяет моделировать поведение внешних систем при достижении ими предельных порогов производительности.

Однако при увеличении масштаба информационной системы количество необходимых заглушек возрастает, что порождает проблему управления распределённой инфраструктурой. Процессы развёртывания, контроля состояния (активна/остановлена) и маршрутизации трафика к десяткам имитационных сервисов на различных серверах в ручном режиме существенно замедляют подготовку к испытаниям.

Таким образом, для эффективного функционирования стендов нагрузочного тестирования необходима специализированная система. Она должна централизованно управлять жизненным циклом заглушек, обеспечивать их мониторинг и динамическое проксирование запросов, что особенно актуально в условиях перехода на отечественные операционные системы, такие как ОС Astra Linux.

1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения

Выбор Java в качестве платформы для реализации имитационных сервисов обусловлен рядом практических факторов. Во-первых, Java является доминирующим языком корпоративной разработки в российской ИТ-отрасли: значительная часть тестируемых систем построена на этой платфор-

ме, что делает естественным использование той же среды исполнения для их имитации. Во-вторых, в реальных проектах заглушки нередко поставляются командами разработки в виде готовых бинарных артефактов (.jar, .war), не требующих модификации перед развёртыванием. В-третьих, формат Fat JAR со встроенным HTTP-сервером стал индустриальным стандартом для микросервисов, что позволяет запускать заглушку как самодостаточный процесс без внешних зависимостей. Таким образом, поддержка Java-артефактов является не проектным решением, а предусловием задачи, продиктованным сложившейся практикой разработки и эксплуатации в предметной области.

Традиционным подходом к управлению имитационными сервисами (заглушками), разработанными на языке Java, является их развёртывание в специализированных контейнерах серверов приложений. В современной практике разработки и тестирования сложился ряд стандартных решений, среди которых наиболее распространёнными являются:

1. Apache Tomcat – наиболее лёгкий и популярный сервлет-контейнер, используемый для запуска веб-приложений. Он обеспечивает базовую поддержку спецификаций Java Servlet и JavaServer Pages.

2. WildFly (ранее JBoss Application Server) – более мощная и полнофункциональная платформа, поддерживающая широкий спектр стандартов Java EE (Jakarta EE). Данное решение предоставляет расширенные возможности управления ресурсами и кластеризации [6].

3. Eclipse GlassFish – эталонная реализация платформы Java EE, предлагающая развитые инструменты администрирования через графическую консоль и командную строку [7].

Анализ данных решений позволяет выделить общие тенденции и характеристики классических серверов приложений:

1. Универсальность и полнофункциональность: все перечисленные платформы спроектированы для запуска сложных корпоративных информационных систем. Однако для узкоспециализированной задачи – работы лёгковесных имитационных заглушек – их функциональность является избы-

точной. Развёртывание подобных платформ сопряжено с настройкой десятков параметров, большинство из которых не применяются в сценариях имитационного тестирования.

2. Значительные накладные расходы: общим недостатком является высокое потребление оперативной памяти и ресурсов центрального процессора самой платформой. При необходимости запуска десятков или сотен загрузок на одном или нескольких хостах суммарные затраты ресурсов на поддержание среды выполнения становятся критическими.

3. Сложность автоматизации и оркестрации: управление большим парком разрозненных экземпляров серверов приложений требует внедрения дополнительных тяжёлых систем автоматизации. Штатные средства управления (консоли администрирования) ориентированы на ручное конфигурирование, что затрудняет их использование в динамических сценариях нагрузочного тестирования.

4. Монолитная архитектура управления: классические подходы подразумевают жёсткую привязку приложения к конкретной конфигурации сервера, что снижает гибкость при необходимости быстрого изменения параметров запуска (например, JVM-аргументов (Java Virtual Machine, виртуальная машина Java)) для отдельных сервисов.

На текущий момент в российской ИТ-индустрии наблюдается отчётливая тенденция к отказу от использования зарубежных проприетарных и открытых решений в пользу отечественного программного обеспечения или собственных разработок. Это обусловлено политикой импортозамещения и необходимостью обеспечения технологического суверенитета.

Большинство западных систем управления инфраструктурой и серверов приложений (таких как продукты Oracle или Red Hat) сталкиваются с проблемами поддержки, обновления и совместимости с российскими операционными системами. В частности, эксплуатация имитационных стендов в защищённой среде ОС Astra Linux требует инструментов, которые нативно

поддерживают механизмы управления этой операционной системы и не зависят от зарубежных репозиторий и лицензий [3].

Таким образом, анализ существующих подходов показывает, что использование традиционных серверов приложений типа Tomcat или WildFly для управления распределённой сетью заглушек становится неэффективным. Современные требования диктуют необходимость перехода к более лёгким, гибким и отечественным решениям, способным обеспечить централизованное управление жизненным циклом сервисов в рамках распределённых кластеров.

1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека

Выбор программных средств для реализации системы управления имитационными сервисами является фундаментальным этапом проектирования. От корректности этого выбора зависят эксплуатационные характеристики будущей системы: предельная пропускная способность, время отклика, масштабируемость и устойчивость к пиковым нагрузкам.

В ходе анализа предметной области и целей исследования было определено, что разрабатываемая система должна обеспечивать бесперебойную оркестрацию интенсивного потока запросов от нагрузочных станций. Это накладывает жёсткие ограничения на архитектуру: управляющий контур не должен становиться «узким местом» при маршрутизации трафика. Требование минимизации накладных расходов исключает использование технологий, склонных к блокировке вычислительных ресурсов при ожидании ответа от внешних систем.

Для обоснования выбора проводится многокритериальный анализ по трём направлениям: язык программирования, веб-фреймворк и система управления базами данных.

Для разработки серверной части системы рассматривались три языка программирования, наиболее востребованных в сфере высоконагруженных систем и системной автоматизации.

Java – объектно-ориентированный язык программирования, являющийся отраслевым стандартом для корпоративных систем и «родной» средой для запуска самих имитационных сервисов (.jar).

Преимущества: строгая статическая типизация, снижающая количество ошибок на этапе компиляции; развитая экосистема библиотек и инструментов профилирования; поддержка полноценной многопоточности (multithreading).

Недостатки: высокая ресурсоёмкость JVM, требующая значительного объёма оперативной памяти; длительное время «холодного старта», что критично для динамических сред тестирования; громоздкий синтаксис для простых задач взаимодействия с процессами операционной системы.

Go (Golang) – компилируемый многопоточный язык программирования, разработанный Google, ориентированный на создание эффективных сетевых сервисов.

Преимущества: высочайшая производительность, сопоставимая с C++, благодаря компиляции в нативный код; эффективная модель конкурентности (goroutines), позволяющая обрабатывать тысячи соединений с минимальными накладными расходами; отсутствие внешних зависимостей (компиляция в единый бинарный файл).

Недостатки: отсутствие гибкости при работе с динамическими структурами данных (JSON (JavaScript Object Notation) произвольной структуры) из-за строгой типизации; специфическая обработка ошибок и отсутствие исключений, что увеличивает объём шаблонного кода; более высокий порог входа по сравнению с интерпретируемыми языками.

Python – высокоуровневый интерпретируемый язык программирования общего назначения с динамической строгой типизацией.

Преимущества: наличие мощных инструментов асинхронного программирования (библиотека `asyncio` [8]), позволяющих эффективно обрабатывать задачи, ограниченные скоростью ввода-вывода (I/O-bound), такие как сетевые запросы или ожидание ответа от базы данных [9]; глубокая интеграция с Linux: модуль `subprocess` предоставляет удобный интерфейс для управления процессами, сигналами и потоками ввода-вывода; высокая скорость разработки благодаря лаконичному синтаксису.

Недостатки: производительность интерпретатора ниже, чем у компилируемых языков; наличие Global Interpreter Lock (GIL), ограничивающего выполнение потоков на одном ядре CPU (Central Processing Unit, центральный процессор) (нивелируется использованием асинхронного подхода) [8,10,11].

Вывод: в качестве основного языка выбран Python. Возможность использования асинхронного программирования в сочетании с нативными средствами управления процессами Linux позволяет обеспечить необходимую производительность, сохраняя гибкость и скорость разработки.

Определив язык реализации, перейдём к следующему направлению анализа – выбору веб-фреймворка. Учитывая необходимость обеспечения высокой пропускной способности системы, выбор архитектуры веб-фреймворка является критическим. Рассматривались три кандидата: Django, Flask и FastAPI.

Django – полнофункциональный синхронный фреймворк корпоративного уровня, следующий принципу «Batteries included» (всё включено).

Преимущества: наличие встроенной панели администрирования, ORM (Object-Relational Mapping) и системы аутентификации «из коробки»; развитая экосистема плагинов и высокий уровень безопасности; строгая структура проекта, облегчающая поддержку крупных монолитных приложений.

Недостатки: монолитная архитектура и избыточный функционал создают значительные накладные расходы на каждый запрос; синхронная модель обработки запросов плохо подходит для высоконагруженных микросерви-

сов, так как блокирует потоки при ожидании ввода-вывода; сложность отключения неиспользуемых компонентов для облегчения системы.

Flask – лёгковесный микрофреймворк, классическое решение, основанное на спецификации WSGI (Web Server Gateway Interface) [12].

Преимущества: минимализм и модульность: разработчик подключает только необходимые библиотеки; низкий порог входа и простота создания простых сервисов; гибкость в выборе архитектурных паттернов.

Недостатки: использование синхронной (блокирующей) модели ввода-вывода – при высокой нагрузке это приводит к исчерпанию пула потоков сервера и росту задержек; отсутствие встроенных механизмов валидации данных (требует подключения сторонних расширений); снижение производительности при большом количестве одновременных соединений [13].

FastAPI [14,15] – современный асинхронный фреймворк, построенный на спецификации ASGI (Asynchronous Server Gateway Interface) [16].

Преимущества: полная поддержка асинхронности (async/await) и событийного цикла, что позволяет обрабатывать тысячи запросов в секунду на одном процессе; высокая производительность благодаря использованию Starlette и uvloop, сопоставимая с Go и Node.js; встроенная быстрая валидация данных и сериализация на базе Pydantic [17].

Недостатки: относительно молодая экосистема по сравнению с Django и Flask; требует понимания принципов асинхронного программирования.

Вывод: выбран фреймворк FastAPI [14,18]. Это единственное решение в экосистеме Python, способное обеспечить стабильную обработку интенсивного трафика с минимальными задержками благодаря асинхронной неблокирующей архитектуре.

Завершив выбор языка и фреймворка, обратимся к третьему направлению анализа – системе управления данными. Для обеспечения высокой отзывчивости системы подсистема хранения данных должна обладать минимальным временем отклика на чтение конфигураций и запись статусов. Сравнивались конкретные распространённые решения.

PostgreSQL – объектно-реляционная СУБД с открытым исходным кодом [19].

Преимущества: гарантия целостности данных (ACID – Atomicity, Consistency, Isolation, Durability) и надёжность хранения; мощный язык запросов SQL (Structured Query Language) и поддержка сложных выборок (JOIN); широкие возможности расширяемости (типы данных, индексы).

Недостатки: накладные расходы на дисковые операции ввода-вывода и журналирование (WAL – Write-Ahead Log, журнал с упреждающей записью) замедляют отклик; избыточность функционала для хранения простых пар «ключ-значение» (конфигурации заглушек); потребность в строгом проектировании схемы данных (Schema migrations).

MySQL – реляционная СУБД, популярная база данных, часто используемая в веб-приложениях.

Преимущества: высокая скорость операций чтения в простых сценариях; широкая распространённость и простота администрирования; поддержка различных движков хранения (Storage Engines).

Недостатки: как и любая реляционная СУБД, зависит от скорости дисковой подсистемы; блокировки таблиц или строк могут снижать конкурентность записи при высокой нагрузке; жёсткость схемы данных.

MongoDB – документоориентированная NoSQL СУБД, система, хранящая данные в формате BSON (бинарный JSON) [20].

Преимущества: гибкая схема данных (Schemaless), идеально подходящая для хранения JSON-конфигураций; горизонтальная масштабируемость (шардинг) из коробки.

Недостатки: потребляет значительно больше памяти и дискового пространства по сравнению с аналогами; в стандартной конфигурации уступает In-Memory решениям по скорости отклика; слабые гарантии транзакционности в старых версиях.

Redis – In-Memory хранилище структур данных [21,22], высокопроизводительная СУБД, работающая полностью в оперативной памяти.

Преимущества: экстремально низкое время доступа (менее 1 мс) благодаря отсутствию обращений к диску при чтении; простая модель данных «Ключ-Значение», полностью соответствующая задаче хранения состояний процессов; нативная поддержка механизма Time-To-Live (TTL) для временных данных.

Недостатки: зависимость объёма хранимых данных от объёма оперативной памяти; риск потери данных при аварийном отключении питания (решается настройкой персистентности).

Вывод: выбрана СУБД Redis [21]. Использование In-Memory архитектуры обеспечивает максимальное быстродействие. Проблема сохранности данных решается включением режима AOF (Append Only File) [23,24], который асинхронно сохраняет операции на диск, гарантируя восстановление конфигураций после перезагрузки без ущерба для производительности.

На основании проведённого анализа и выявленных требований к быстродействию и надёжности утверждён следующий стек технологий для реализации выпускной квалификационной работы: язык программирования Python с использованием asyncio, веб-фреймворк FastAPI на сервере приложений Uvicorn, СУБД Redis в режиме персистентности AOF и целевая платформа ОС Astra Linux Special Edition [25].

Данный выбор обеспечивает оптимальный баланс между производительностью, скоростью разработки и надёжностью функционирования системы в условиях высокой нагрузки.

1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса

В рамках анализа предметной области необходимо рассмотреть существующие на рынке инструменты, применяемые для создания, эксплуатации и управления имитационными сервисами (заглушками). Поскольку разрабатываемая система предназначена для управления инфраструктурой заглушек в процессе нагрузочного тестирования, к аналогам следует отнести

программные продукты, обеспечивающие виртуализацию сервисов (Service Virtualization), создание mock-сервисов (Mocking), а также серверы приложений, традиционно используемые для хостинга Java-артефактов.

Сравнительный анализ проводится по критериям, критически важным для обеспечения стабильности нагрузочного стенда и автоматизации процессов тестирования:

1. Архитектура и потребление ресурсов: способность работать в условиях ограниченных аппаратных мощностей при запуске десятков экземпляров заглушек.

2. Возможности управления процессами (Orchestration): наличие инструментов для удалённого запуска, остановки и обновления версий приложений-заглушек.

3. Функциональность управления трафиком: наличие встроенных механизмов ограничения нагрузки и эмуляции сетевых задержек.

4. Совместимость и импортозамещение: возможность автономной работы в среде ОС Astra Linux без зависимости от зарубежных облачных сервисов и проприетарных лицензий.

Рассмотрим каждый из аналогов в соответствии с обозначенными критериями.

WireMock – один из наиболее распространённых инструментов с открытым исходным кодом для виртуализации HTTP-сервисов. Продукт позволяет создавать заглушки, настраивать правила сопоставления запросов (request matching) и формировать динамические ответы.

Согласно официальной документации [26], WireMock поддерживает гибкую настройку ответов через JSON API, имитацию задержек (network latency) и ошибок (fault injection). Он может работать в двух режимах: как библиотека, встраиваемая в код автотестов, или как отдельный процесс (Standalone). Помимо этого, инструмент поддерживает запись реальных взаимодействий (record and playback), что позволяет автоматически создавать заглушки на основе перехваченного трафика.

Недостатки в контексте поставленной задачи:

1. Ресурсоёмкость: WireMock разработан на Java и выполняется в виртуальной машине JVM. Запуск отдельного экземпляра WireMock для каждого имитируемого микросервиса создаёт значительные накладные расходы на оперативную память (RAM) и процессорное время (CPU). В условиях нагрузочного тестирования, когда требуется поднять 50–100 заглушек, суммарные затраты ресурсов на поддержание работы самих заглушек становятся неприемлемыми.

2. Отсутствие встроенной оркестрации: в бесплатной версии (Open Source) WireMock отсутствует централизованная панель управления распределённым кластером. Инструмент фокусируется на логике ответа, а не на эксплуатации сервера.

3. Ограничения Rate Limiting: базовый функционал позволяет имитировать задержки, однако полноценный алгоритм ограничения количества запросов в секунду (RPS – Requests Per Second) с отбрасыванием лишнего трафика требует написания пользовательских расширений на Java [26].

MockServer – популярное решение для создания mock-сервисов, взаимодействующих по протоколам HTTP или HTTPS (HTTP Secure). Продукт часто используется в контейнеризированных средах (Docker, Kubernetes) для интеграционного тестирования.

MockServer предоставляет мощный API для создания «ожиданий» (Expectations) и верификации запросов. Согласно документации [27], он поддерживает проксирование трафика (Port Forwarding), запись проходящих запросов для последующего анализа и генерацию ответов на основе шаблонов (Mustache, Velocity).

Недостатки в контексте поставленной задачи:

1. Монолитная архитектура управления: MockServer хранит состояния «ожиданий» в памяти конкретного экземпляра приложения. Для создания распределённой системы требуется сложная настройка кластеризации, что утяжеляет инфраструктуру тестового стенда.

2. Сложность динамического управления: MockServer ориентирован на программное управление через API во время прогона тестов. Использование его как постоянно действующего инфраструктурного компонента требует разработки дополнительной обвязки для мониторинга его состояния и автоматического перезапуска [27].

3. Зависимость от Java: аналогично WireMock, продукт написан на Java, что влечёт проблемы с «холодным стартом» и высоким потреблением памяти (Heap Size), снижающим плотность размещения заглушек на сервере.

Mountebank – инструмент с открытым исходным кодом, разработанный на платформе Node.js. Его ключевой особенностью является мультипротокольность: помимо HTTP/HTTPS, он поддерживает TCP (Transmission Control Protocol), SMTP (Simple Mail Transfer Protocol) и другие протоколы через систему плагинов.

Mountebank использует концепцию «самозванцев» (imposters) – лёгких сервисов, слушающих определённые порты. Документация [28] указывает на возможность использования JavaScript-инъекций для реализации сложной логики обработки ответов.

Недостатки в контексте поставленной задачи:

1. Проблемы безопасности: возможность исполнения произвольного JS-кода (injection) в теле запроса создаёт уязвимости в контуре нагрузочного тестирования.

2. Отсутствие управления артефактами: Mountebank позволяет создать логику ответа (скрипт), но не предназначен для запуска и управления бинарными файлами (заглушками в виде .jar), предоставленными разработчиками.

3. Производительность в однопоточном режиме: в стандартной конфигурации Mountebank работает в одном потоке (Event Loop). При высокой нагрузке (тысячи RPS), характерной для нагрузочного тестирования, сложные вычисления внутри одной заглушки могут заблокировать обработку запросов других заглушек, работающих в том же экземпляре Mountebank [28].

Наряду с инструментами виртуализации сервисов необходимо оценить традиционные серверы приложений, также применяемые для хостинга Java-артефактов.

Apache Tomcat – свободно распространяемый контейнер сервлетов, реализующий спецификации Jakarta Servlet и Jakarta Pages. Это классическое решение для хостинга Java-приложений, широко применяемое в промышленной эксплуатации.

Согласно официальной документации [29], Tomcat предоставляет развитые средства управления жизненным циклом веб-приложений. Компонент Manager App позволяет развёртывать, останавливать и перезапускать приложения без остановки сервера. Поддерживается кластеризация для репликации сессий [30]. Для защиты от перегрузок предусмотрен фильтр RateLimitFilter, позволяющий ограничивать количество запросов с одного IP-адреса (Internet Protocol) [31].

Недостатки в контексте поставленной задачи:

1. Избыточность архитектуры: Tomcat является полноценным сервлет-контейнером. Использование его для запуска примитивных заглушек не рационально с точки зрения потребления ресурсов. Запуск отдельного экземпляра Tomcat для каждой заглушки потребляет сотни мегабайт оперативной памяти только на нужды самого сервера, что критично при дефиците ресурсов. Кроме того, время холодного старта Tomcat составляет десятки секунд, тогда как встроенные серверы (Netty, Undertow) запускаются за единицы секунд, что замедляет итерации при массовом развёртывании заглушек.

2. Требования к формату артефактов: Tomcat работает с WAR-архивами (Web Application Archive). Однако современные микросервисы (и заглушки для них) чаще всего поставляются в виде Fat JAR (Java Archive, со встроенным сервером, например, Netty или Undertow). Для запуска таких заглушек в Tomcat требуется их пересборка и модификация кода, что усложняет процесс и вносит риск конфликтов зависимостей между библиотеками заглушки и библиотеками самого контейнера.

3. Статичность конфигурации ограничений: механизмы Rate Limiting в Tomcat конфигурируются декларативно через XML-файлы при старте. В задачах нагрузочного тестирования требуется динамическое изменение параметров дросселирования трафика «на лету» без перезагрузки сервера, что в Tomcat не реализовано «из коробки».

4. Отсутствие унифицированного API управления: Tomcat предоставляет Manager App с HTTP-интерфейсом для развёртывания приложений, однако этот интерфейс не предназначен для программной оркестрации. В сценариях автоматизированного нагрузочного тестирования необходим полноценный API для группового управления жизненным циклом заглушек (запуск, остановка, мониторинг состояния), которого Tomcat не предоставляет без написания дополнительных обёрток.

Для наглядного сравнения рассмотренных решений с проектируемой системой составлена сводная таблица 1. В ней отражены ключевые характеристики каждого из аналогов, что позволяет обоснованно выделить преимущества и недостатки каждого подхода.

Таблица 1 – Сравнительный анализ средств управления имитационными сервисами

Критерий	WireMock	MockServer	Mountebank	Apache Tomcat	Разраб. система
Управление JAR/WAR без пересборки	Нет	Нет	Нет	Частично	Да
Централизованная оркестрация	Нет	Нет	Нет	Нет	Да
Динамический Rate Limiting	Нет	Нет	Нет	Нет	Да
Потребление ресурсов	Высокое (JVM)	Высокое (JVM)	Среднее	Высокое (JVM)	Низкое
Совместимость с Astra Linux	Частично	Частично	Да	Частично	Да

Проведённый анализ показывает, что существующие на рынке решения делятся на две категории. Инструменты логической виртуализации (WireMock, MockServer, Mountebank) отлично справляются с задачей «что

ответить на запрос», но не решают задачу управления инфраструктурой (запуск, перезапуск, обновление версий приложений). Серверы приложений (Apache Tomcat), в свою очередь, умеют управлять жизненным циклом, но слишком тяжеловесны, требуют специфического формата упаковки (WAR) и сложны в динамической настройке сетевых параметров.

Ни одно из рассмотренных решений не предоставляет функционала «лёгковесного агента» для управления сторонними Java-процессами «как есть» (без перепackовки). Выявленный пробел обуславливает создание специализированного программного комплекса.

Разработка собственной автоматизированной информационной системы обоснована следующими факторами:

1. Специфика задачи (Lifecycle Management): разрабатываемая система решает задачу оркестрации процессов операционной системы. Она позволяет загружать бинарный файл заглушки (JAR), полученный от разработчиков, и управлять его исполнением на удалённом сервере. Это исключает трудозатраты на пересборку заглушек под WireMock или Tomcat.

2. Эффективность использования ресурсов: перенос логики управления на Python и использование асинхронного фреймворка FastAPI позволяет минимизировать накладные расходы самого управляющего контура. Система работает поверх заглушек, не внедряясь в их память, в отличие от тяжёлых Java-контейнеров.

3. Централизация и удобство эксплуатации: реализация архитектуры «Master-Agent» объединяет разрозненные серверы в единый кластер. Единая панель управления позволяет инженеру тестирования массово обновлять версии заглушек и менять настройки лимитов (RPS) в реальном времени, что невозможно при использовании Standalone-решений.

4. Технологическая независимость и импортозамещение: система разрабатывается на базе открытых технологий, адаптирована для работы в ОС Astra Linux и не зависит от внешних репозиториев (Maven Central, Docker

Hub) или лицензионных ключей, что гарантирует стабильность работы в закрытых корпоративных контурах РФ.

Разработка собственного программного комплекса является единственным целесообразным решением, закрывающим пробел между инструментами логического создания mock-сервисов и средствами управления инфраструктурой, обеспечивая высокую производительность и гибкость конфигурации тестовых сред. Дополнительным преимуществом является полный контроль над технологическим стеком и возможность адаптации системы под требования закрытых корпоративных контуров без зависимости от сторонних лицензий и внешних репозиторий.

1.5 Формирование функциональных и технических требований к системе

На основании проведённого анализа предметной области и специфики задач нагрузочного тестирования сформированы требования к разрабатываемому программному комплексу [32,33]. Система классифицируется как инструмент автоматизации инфраструктуры нагрузочного тестирования. Архитектура разрабатываемого решения должна быть гибкой, допускающей работу как в распределённом контуре, так и в рамках одного физического или виртуального сервера.

Система должна поддерживать два сценария развёртывания без изменения кодовой базы:

1. Распределённый режим (Cluster Mode): классическая клиент-серверная архитектура, где управляющий узел (Master) администрирует множество удалённых узлов (Agents), на которых выполняются заглушки. Взаимодействие осуществляется по сети.

2. Автономный режим (Standalone Mode): консолидированный запуск всех компонентов системы на одном сервере. В этом режиме управляющий модуль и модуль исполнения заглушек функционируют в пределах одной

операционной системы, обеспечивая изоляцию контура тестирования без необходимости сетевого взаимодействия между узлами инфраструктуры.

Требования к подсистеме управления жизненным циклом:

1. Поддержка форматов артефактов: система должна обеспечивать запуск Java-приложений, поставляемых в форматах исполняемых файлов .jar и веб-архивов .war.

2. Управление процессами: обеспечение запуска, корректной остановки и принудительного завершения процессов заглушек.

3. Конфигурация запуска: возможность задания аргументов виртуальной машины (JVM Options), включая параметры памяти (-Xms, -Xmx) и системные свойства, через графический интерфейс управляющего узла (Master) перед стартом заглушки.

4. Мониторинг: отслеживание статуса процесса (PID – Process Identifier, идентификатор процесса) и доступности его сетевого порта.

5. Работа с логами: сбор стандартных потоков вывода (stdout, stderr) запущенных процессов и их трансляция в интерфейс системы для отладки.

Система должна работать как прозрачный прокси-сервер (Transparent Proxy) на прикладном уровне, реализуя следующий алгоритм:

1. Приём входящего HTTP-запроса от внешней системы.

2. Извлечение метаданных: HTTP-метод, заголовки, тело запроса и целевой путь (URI – Uniform Resource Identifier, унифицированный идентификатор ресурса).

3. Выполнение нового запроса к целевой заглушке (на её локальный порт) с использованием извлечённого метода, заголовков и тела.

4. Получение ответа от заглушки.

5. Возврат полученного ответа инициатору запроса в неизменном виде.

Требования к управлению нагрузкой (Rate Limiting):

1. Наличие механизма ограничения количества запросов в секунду (RPS) для конкретной заглушки.

2. Активация лимита производится через вызов конфигурационного маршрута системы (API – Application Programming Interface, интерфейс прикладного программирования).

3. При превышении лимита система должна возвращать HTTP-код 429 (Too Many Requests), не передавая запрос заглушке.

Требования к управляющему модулю:

1. Управление артефактами: загрузка бинарных файлов (.jar, .war) в хранилище системы через веб-интерфейс.

2. Реестр сервисов: отображение списка всех загруженных заглушек, их текущего статуса («Запущен», «Остановлен», «Ошибка») и адресов.

3. Интерфейс конфигурации: предоставление полей ввода для настройки параметров запуска (JVM args) для каждого сервиса.

Требования к производительности сформированы на основании ГОСТ Р ИСО/МЭК [34]. Накладные расходы на проксирование (вносимая задержка) не должны превышать 20 мс. Снижение максимальной пропускной способности (RPS) заглушки при работе через систему не должно превышать 10 % относительно прямого обращения к заглушке.

Требования к надёжности включают два аспекта. Во-первых, при перезапуске системы (или сервера) должны сохраняться настройки запуска заглушек и ссылки на загруженные артефакты (Persistence); для заглушек, находившихся в активном состоянии до перезапуска, система должна предпринять автоматическую попытку повторного запуска процесса. Во-вторых, ошибка в одной заглушке не должна влиять на работоспособность остальных запущенных сервисов и самой управляющей системы (изоляция сбоев).

Системные требования предусматривают функционирование серверной части под управлением ОС Astra Linux Special Edition [35] (версия 1.7 и выше) или аналогичных Linux-дистрибутивов, а также наличие OpenJDK 17 [36] для запуска пользовательских артефактов.

Стек разработки включает язык программирования Python версии 3.10 и выше с использованием асинхронного подхода, веб-фреймворк FastAPI для

реализации высокопроизводительного ввода-вывода и СУБД Redis для хранения конфигураций и реализации счётчиков Rate Limiter.

Загружаемые исполняемые файлы должны храниться в изолированных директориях с правами доступа, исключающими их несанкционированную модификацию третьими лицами.

1.6 Выводы

В первой главе выпускной квалификационной работы был проведён комплексный анализ предметной области и обоснована необходимость создания специализированного инструментария для автоматизации нагрузочного тестирования. В ходе исследования выявлено, что использование традиционных серверов приложений и существующих аналогов (WireMock, MockServer, Mountebank) для управления распределённой сетью заглушек сопряжено с избыточным потреблением ресурсов и сложностями в эксплуатации, а также не удовлетворяет требованиям по импортозамещению.

На основании сравнительного анализа обоснован выбор технологического стека: язык программирования Python с использованием асинхронного подхода, веб-фреймворк FastAPI для обеспечения высокой пропускной способности и In-Memory СУБД Redis для хранения состояний с минимальной задержкой.

Определена монолитная архитектура будущего решения с управлением удалёнными хостами через asyncssh/SFTP (SSH File Transfer Protocol) [37] и сформированы функциональные и технические требования к системе, включая поддержку форматов .jar и .war, механизмы ограничения нагрузки (Rate Limiting) и совместимость с ОС Astra Linux [38]. Полученные результаты служат базисом для проектирования и технической реализации системы во второй главе.

2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ

2.1 Разработка архитектуры распределённого программного комплекса

Для разрабатываемого комплекса рассматривались два архитектурных стиля: микросервисный и монолитный. Микросервисная архитектура неоправданно усложнила бы систему: потребовала бы оркестратора контейнеров (внешняя зависимость, противоречащая требованию технологической независимости), ввела сетевые задержки между внутренними компонентами и увеличила операционную нагрузку при разработке одним специалистом. Разрабатываемая система является «тонким» координирующим слоем над имитационными сервисами, поэтому накладные расходы микросервисной декомпозиции несоразмерны её выгодам.

Выбрана монолитная архитектура с явно выраженной модульной структурой. Сравнительный анализ подходов приведён в таблице 2.

Таблица 2 – Сравнение архитектурных подходов

Критерий	Монолитная архитектура	Микросервисная архитектура
Накладные расходы	Минимальные	Высокие
Сложность развёртывания	Низкая	Высокая
Зависимость от внешних систем	Отсутствует	Требуется оркестратор
Сопровождаемость	Высокая	Низкая
Масштабирование компонентов	Не поддерживается	Поддерживается

Несмотря на монолитную организацию, распределённость системы реализуется на уровне топологии развёртывания, а не внутренней декомпозиции: единый управляющий процесс координирует целевые хосты через реестр. Каждый хост реестра описывается сетевым адресом, учётной записью SSH и рабочей директорией. При регистрации заглушки оператор выбирает целевой хост, что позволяет совмещать локальное (127.0.0.1) и удалённое (asyncssh/SFTP) размещение в рамках одной инфраструктуры без изменения

конфигурации приложения. Компонент Host Resolver предоставляет функцию `is_local(hostname)`, инкапсулируя точку ветвления между локальными системными вызовами и SSH-операциями.

Для формализованного описания архитектуры применяется нотация C4 Model [39,40]. Контекстная диаграмма (рисунок 1) представляет систему в окружении внешних акторов: инженера тестирования (управляет инфраструктурой через веб-интерфейс), потребителя (HTTP-клиент: нагрузочный инструмент или тестируемая система), Prometheus (сбор метрик) и целевого хоста Astra Linux [25] с Java-процессами [36].

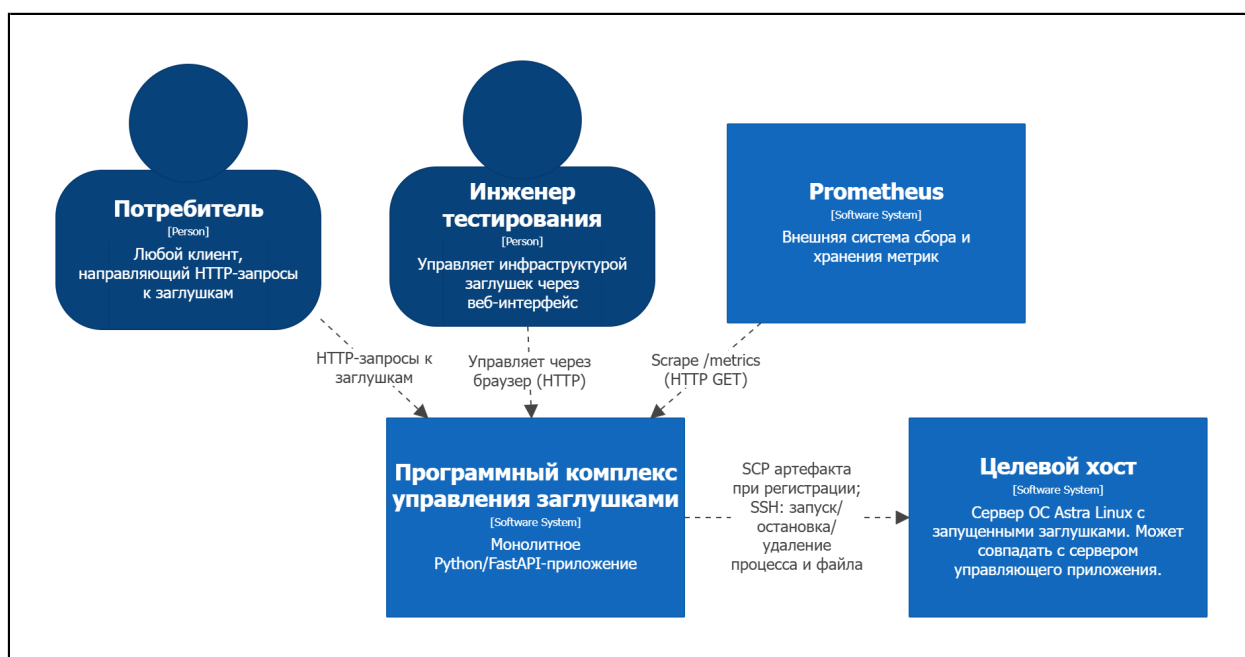


Рисунок 1 – Контекстная диаграмма системы (C4 Level 1)

Контейнерная диаграмма (рисунок 2) раскрывает три программных контейнера системы. Веб-интерфейс (FastAPI / Jinja2 SSR (Server-Side Rendering, отрисовка на стороне сервера) [41]) формирует HTML-страницы (HyperText Markup Language) без внешних CDN (Content Delivery Network, сеть доставки контента). REST API (Python / FastAPI / Uvicorn) реализует всю бизнес-логику. Redis (AOF-персистентность) хранит конфигурации и счётчики Rate Limiter. Хранилище артефактов намеренно не предусмотрено: файл заглушки копируется на целевой хост по SFTP и удаляется.

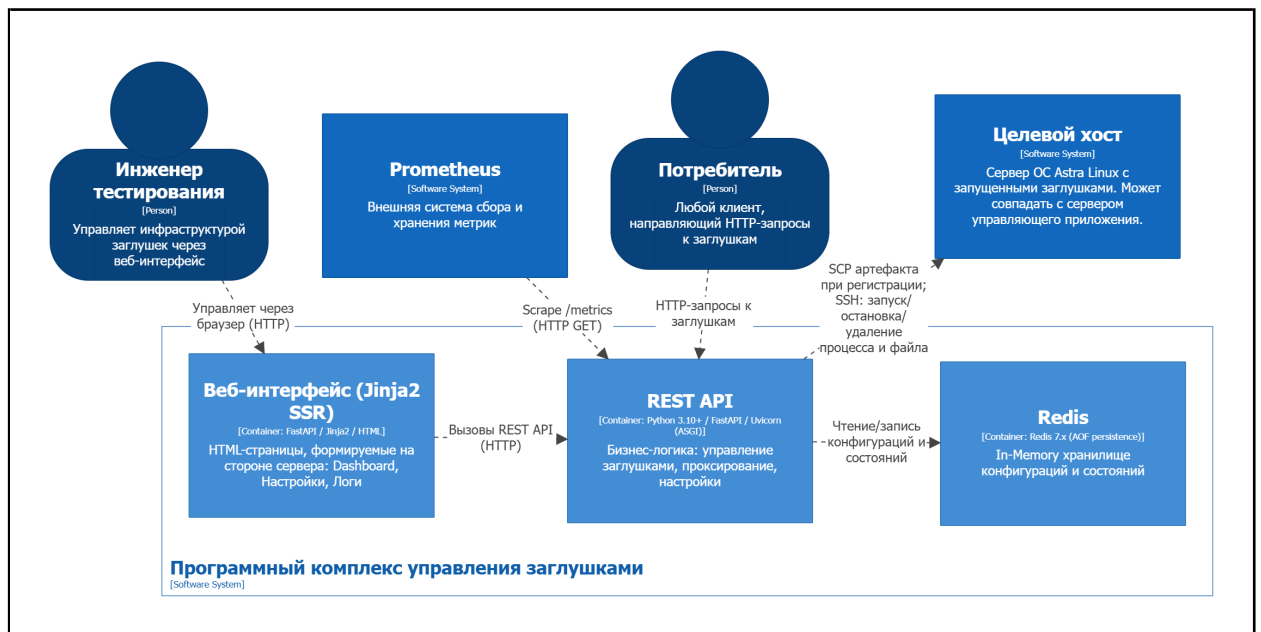


Рисунок 2 – Контейнерная диаграмма системы (C4 Level 2)

Компонентная диаграмма (рисунок 3) детализирует внутреннюю структуру REST (Representational State Transfer) API. Назначение шести компонентов приведено в таблице 3.

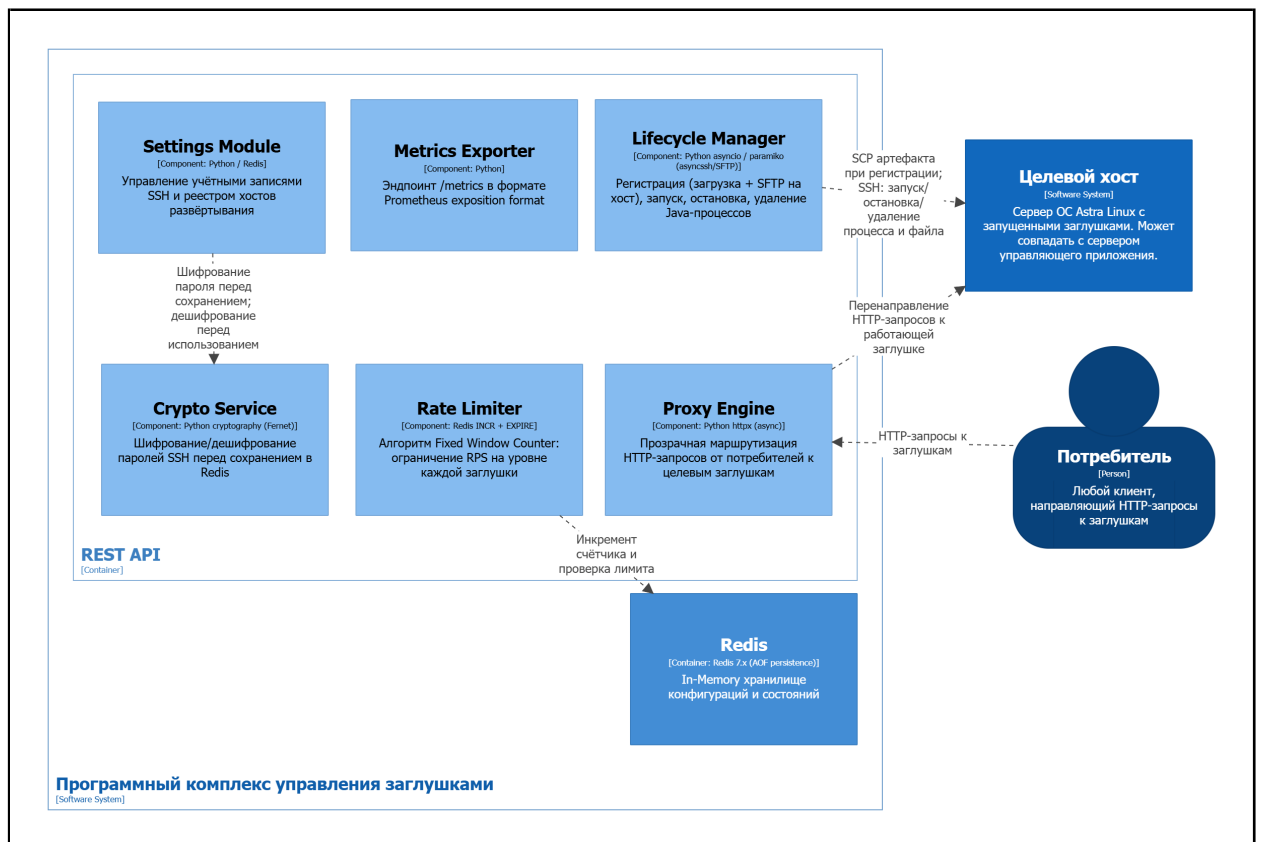


Таблица 3 – Компоненты REST API

Компонент	Назначение
Lifecycle Manager	Регистрация (SFTP-загрузка на хост [42]), запуск (покуп через SSH), остановка и удаление Java-процессов
Proxy Engine	Прозрачная маршрутизация HTTP-запросов к заглушкам через httpx [43] с сохранением метода, заголовков и тела
Rate Limiter	Алгоритм Fixed Window Counter на уровне каждой заглушки; управляется флагом rate_limit_enabled без перезапуска
Settings Module	Управление учётными записями SSH и реестром хостов развёртывания
Crypto Service	Шифрование/дешифрование паролей SSH алгоритмом AES-128 (Advanced Encryption Standard) (Fernet [44]); ключ хранится в файле с правами 600
Metrics Exporter	Маршрут /metrics в формате Prometheus exposition format [45–47]; детали в разделе 2.6

Диаграмма развёртывания (рисунок 4) показывает два типа узлов: управляющий сервер и произвольное число целевых хостов. Lifecycle Manager взаимодействует с каждым хостом через единое SSH-соединение: по нему передаётся JAR-артефакт заглушки по SFTP-подпротоколу, а также выполняются команды запуска и остановки Java-процессов.

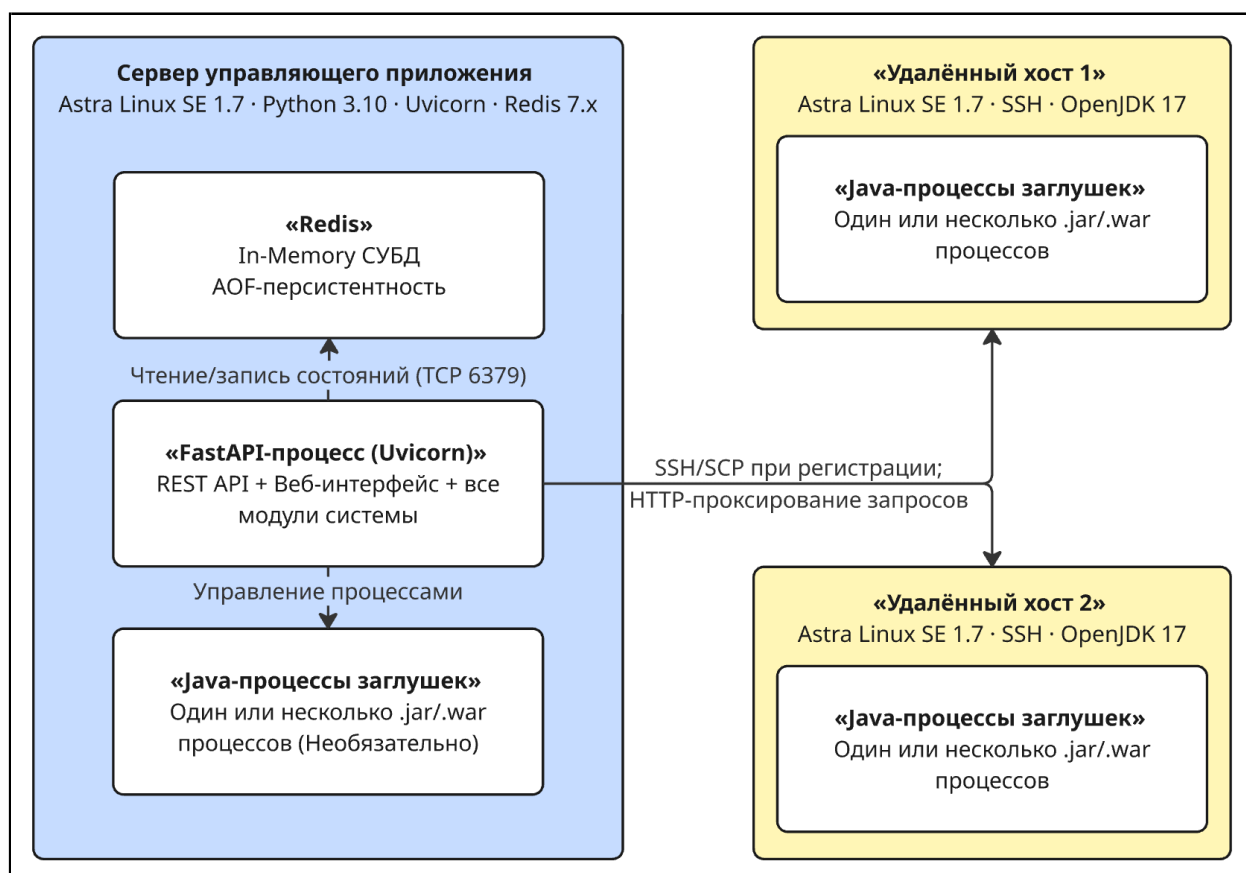


Рисунок 4 – Диаграмма развёртывания

Для осуществления SSH-операций, отражённых на диаграмме развёртывания, управляющее приложение должно хранить учётные данные целевых хостов. Этой задачей занимается модуль настроек (Settings Module), управляющий двумя категориями конфигурационных данных. Учётные записи ОС Linux используются для SSH-аутентификации на целевых хостах; применение учётных записей с ограниченными правами минимизирует риски при нештатном поведении Java-процессов. Пароли хранятся в Redis в зашифрованном виде: Crypto Service применяет AES-128 (Fernet [44]), ключ хранится вне Redis в файле с правами 600; дешифрование выполняется только в оперативной памяти на время установки соединения.

Реестр хостов представляет собой список серверов для развёртывания заглушек. Параметры записи приведены в таблице 4.

Таблица 4 – Параметры модуля настроек системы

Параметр	Тип	Описание
hostname	String	IP-адрес или имя хоста; уникальный идентификатор
ssh_port	Integer	Порт SSH (по умолчанию 22)
account_uuid	UUID (Universally Unique Identifier)	ID учётной записи SSH из реестра accounts
working_dir	String	Абсолютный путь рабочей директории на хосте
java_path	String	Путь к исполняемому файлу Java (явный, т.к. JAVA_HOME в корп. средах нередко не задан)
mock_port_min	Integer	Нижняя граница диапазона TCP-портов для заглушек
mock_port_max	Integer	Верхняя граница диапазона TCP-портов для заглушек
description	String	Текстовое описание хоста

Диапазон mock_port_min/mock_port_max обеспечивает совместимость с политиками межсетевого экрана в закрытых корпоративных сетях, где разрешается трафик только на заранее согласованные порты.

В таблице 5 приведены пять типовых сценариев, иллюстрирующих сквозное взаимодействие компонентов. Ключевое архитектурное решение:

бинарный артефакт копируется на целевой хост однократно при регистрации; все последующие операции работают с файлом на хосте.

Таблица 5 – Типовые сценарии взаимодействия компонентов

Сценарий	Последовательность действий
Регистрация заглушки	Загрузка файла → временный каталог на сервере → SFTP на целевой хост → удаление временного файла → HSET <code>mocks:{filename}</code> + SADD <code>mocks:registry</code> (один Pipeline), статус REGISTERED
Запуск заглушки	Считать конфигурацию из Redis → дешифровать пароль → определить свободный порт через SSH → <code>nohup {java_path} {jvm_args} -jar {artifact_path} --server.port={port}</code> → сохранить PID, порт, статус RUNNING в Redis
Остановка заглушки	Считать PID из Redis → выполнить <code>kill {PID}</code> через SSH (SIGTERM; при отсутствии реакции – SIGKILL) → HSET <code>mocks:filename status STOPPED</code> , поля <code>pid</code> и <code>port</code> очищаются
Обработка запроса	Считать статус и флаг <code>rate_limit_enabled</code> из Redis → при статусе $\neq$ RUNNING: HTTP 503 → при включённом Rate Limiter: <code>INCR rate:{mock}:{window}</code> , при превышении: HTTP 429 → иначе: проксировать через <code>httpx</code> , вернуть ответ без изменений
Переключение Rate Limiter	<code>PATCH /api/v1/mocks/{name}/rate-limit</code> → атомарный HSET <code>rate_limit_enabled</code> в Hash-записи заглушки → применяется со следующего запроса без перезапуска
Удаление заглушки	Остановить процесс через SSH (SIGTERM / SIGKILL) → <code>SSH rm {artifact_path}</code> → <code>DEL mocks:{filename}</code> + <code>SREM mocks:registry</code>

2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis

Redis выполняет роль единственного хранилища всех оперативных данных системы: конфигураций заглушек, параметров хостов, учётных записей, состояний процессов, счётчиков Rate Limiter и реестров объектов. Бинарные файлы артефактов в Redis не хранятся – единственной постоянной копией является файл на целевом хосте. Выбор Redis обусловлен временем доступа менее 1 мс, критически важным для Rate Limiter, выполняемого при обработке каждого входящего запроса.

Для хранения конфигурационных объектов (заклушки, хосты, учётные записи, глобальные настройки) используется тип Hash: каждый атрибут объ-

екта хранится как отдельное поле, что позволяет обновлять конкретные поля командой HSET без перезаписи всей записи и читать объект целиком командой HGETALL за одну операцию. Для реестров идентификаторов применяется тип Set, гарантирующий уникальность. Для счётчиков Rate Limiter – тип String с атомарными командами INCR и EXPIRE.

В таблице 6 приведена сводная схема пространств имён ключей.

Таблица 6 – Пространства имён ключей Redis

Ключ	Тип	Назначение
mocks:{filename}	Hash	Конфигурация и состояние заглушки
mocks:registry	Set	Множество имён файлов всех заглушек
hosts:{hostname}	Hash	Конфигурация целевого хоста
hosts:registry	Set	Множество hostname всех хостов
accounts:{uuid}	Hash	Учётная запись SSH
accounts:registry	Set	Множество UUID учётных записей
rate:{filename}:{window}	String	Счётчик Rate Limiter (с TTL)
settings:global	Hash	Глобальные параметры приложения

Структура Hash-записи заглушки (mocks:{filename}) приведена в листинге 2.1. Имя файла является уникальным идентификатором заглушки: при попытке зарегистрировать файл с уже существующим именем система возвращает ошибку. Поле artifact_path вычисляется однократно при регистрации как конкатенация working_dir хоста и имени файла, что исключает повторное обращение к записи хоста при каждой операции. Поля port, pid и started_at заполняются после успешного запуска и сбрасываются при остановке.

Листинг 2.1 – Структура Hash-записи заглушки в Redis

```
1 HSET mocks:{filename}
2   hostname      test-server-01
3   port          8105
4   pid           24731
5   status        RUNNING
6   jvm_args       -Xms128m -Xmx256m
7   rate_limit     500
8   rate_limit_enabled true
9   registered_at  2024-11-01T10:30:00+00:00
10  started_at    2024-11-01T10:35:00+00:00
11  artifact_path  ↪ /opt/mock-services/payment-service-stub.jar
```

Таблица 7 – Поля Hash-записи заглушки

Поле	Тип	Описание
hostname	String	Хост развёртывания заглушки
port	Integer/null	TCP-порт процесса
pid	Integer/null	PID процесса на хосте
status	String	REGISTERED / RUNNING / STOPPED / ERROR
jvm_args	String	Аргументы JVM
rate_limit	Integer	Макс. запросов в секунду
rate_limit_enabled	Boolean	Флаг активности Rate Limiter
registered_at	ISO 8601	Дата регистрации
started_at	ISO 8601/null	Дата последнего запуска
artifact_path	String	Абсолютный путь к JAR/WAR на целевом хосте

Структура Hash-записи хоста (`hosts:{hostname}`) приведена в листинге 2.2. Hostname одновременно является уникальным идентификатором записи и адресом для `asyncssh/SFTP`-подключения, обеспечивая прямой доступ за одну операцию `HGETALL`. Поле `status` отражает результат последней фоновой проверки SSH-доступности (`AVAILABLE` / `UNAVAILABLE` / `UNKNOWN`); поле `account_uuid` указывает на запись `accounts:{uuid}` с зашифрованными учётными данными.

Листинг 2.2 – Структура Hash-записи хоста в Redis

```

1 HSET hosts:{hostname}
2     ssh_port      22
3     account_uuid  e5f6a7b8-c9d0-1234-efab-567890abcdef
4     working_dir   /opt/mock-services
5     java_path     /usr/lib/jvm/java-17-openjdk-amd64/bin/java
6     mock_port_min 8100
7     mock_port_max 8200
8     description   Стенд нагрузочного тестирования.
9     status        AVAILABLE
10    last_checked_at 2024-11-01T10:29:55+00:00

```

Структура Hash-записи учётной записи (`accounts:{uuid}`) приведена в листинге 2.3. Поле `password_enc` содержит Base64-кодированный зашифрованный блок (`Fernet/AES-128`); дешифрование выполняется `Crypto Service` только в оперативной памяти на время установки SSH-соединения – расшифрованный пароль никогда не записывается на диск.

Листинг 2.3 – Структура Hash-записи учётной записи в Redis

```
1 HSET accounts:{uuid}
2     username      mock-runner
3     password_enc   gAAAAAB1...base64-encoded-ciphertext...
4     description    Сервисный пользователь для запуска заглушек
5     created_at     2024-10-15T09:00:00+00:00
```

ER-диаграмма связей объектов в Redis приведена на рисунке 5. Схема включает три ключевые сущности: заглушку (mocks:{uuid}), хост (hosts:{hostname}) и учётную запись (accounts:{uuid}). Заглушка связана с хостом через поле hostname, которое одновременно служит ключом Redis-записи и DNS-именем для SSH-подключения. Хост, в свою очередь, ссылается на учётную запись через поле account_uuid, определяя, какие зашифрованные учётные данные использовать при установке соединения. Таким образом, каждая заглушка транзитивно связана ровно с одной учётной записью через промежуточную запись хоста.

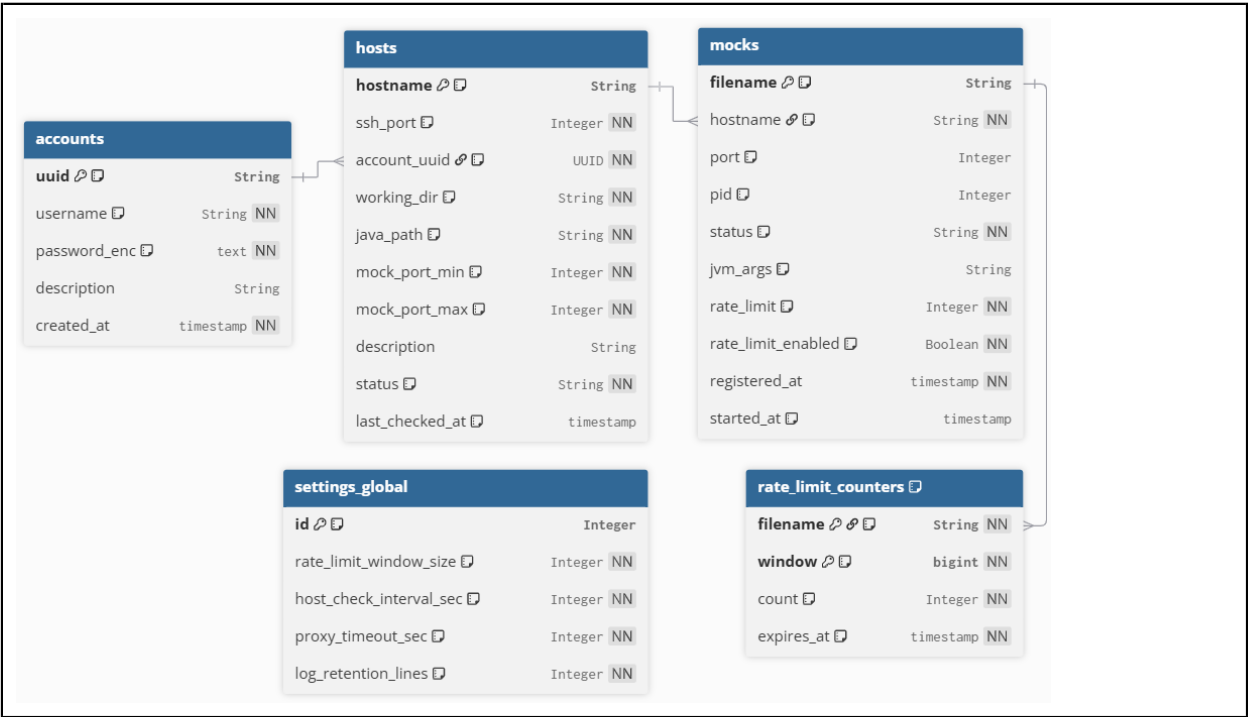


Рисунок 5 – ER-диаграмма объектов в Redis

Помимо постоянных конфигурационных записей, система использует эфемерные ключи для реализации алгоритма ограничения интенсивности запросов. Для каждой заглушки и каждого временного окна в Redis хранится счётчик по ключу rate:{filename}:{window}, где {window} – целочисленное

деление текущего Unix-timestamp на длину окна в секундах. Алгоритм обработки запроса описан в листинге 2.4.

Листинг 2.4 – Псевдокод алгоритма Fixed Window Counter

```
1 def check_rate_limit(filename: str, limit: int, window_size: int =  
    ↪ 1) -> bool:  
2     # Шаг 1: вычислить метку текущего окна  
3     window = floor(current_unix_timestamp() / window_size)  
4     key     = f"rate:{filename}:{window}"  
5     # Шаг 2: атомарный инкремент счётчика  
6     count = INCR(key)  
7     # Шаг 3: установить TTL при создании нового окна  
8     if count == 1:  
9         EXPIRE(key, window_size * 2)  
10    # Шаг 4: проверить лимит  
11    if count > limit:  
12        return False  
13    return True
```

TTL устанавливается равным удвоенному размеру окна и только при первом запросе, что гарантирует самоочистку устаревших ключей. Атомарность INCR [48] обеспечивает корректный подсчёт при конкурентном поступлении запросов без блокировок.

Глобальные параметры приложения, определяющие в том числе размер временного окна Rate Limiter, хранятся под ключом settings:global (Hash). Структура записи приведена в листинге 2.5, описание полей – в таблице 8.

Листинг 2.5 – Структура Hash-записи глобальных настроек в Redis

```
1 HSET settings:global  
2     rate_limit_window_size      1  
3     host_check_interval_sec     30  
4     proxy_timeout_sec           10  
5     log_retention_lines         1000
```

Таблица 8 – Поля Hash-записи глобальных настроек системы

Поле	Тип	Описание
rate_limit_window_size	Integer	Размер окна Rate Limiter (сек)
host_check_interval_sec	Integer	Интервал проверки доступности хостов (сек)
proxy_timeout_sec	Integer	Таймаут проксирования (сек)
log_retention_lines	Integer	Макс. строк лога процесса в буфере

Redis настроен в режиме AOF [23] (appendfsync everysec): каждая операция записи фиксируется в журнале, синхронизируемом с диском раз в секунду; максимальная потеря данных при аварии – не более одной секунды.

При перезапуске управляющего приложения выполняется следующая процедура восстановления: считывается реестр заглушек, затем для каждой заглушки читается полная Hash-запись. Для каждой заглушки со статусом RUNNING выполняется SSH-проверка фактического наличия процесса по сохранённому PID. При подтверждении – статус сохраняется, восстанавливается HTTP-клиент прокси-движка. При отсутствии процесса – предпринимается автоматический перезапуск с сохранёнными JVM-аргументами; в случае успеха запись обновляется новыми pid, port и started_at, при неудаче – ошибка фиксируется в журнале, конфигурация остаётся неизменной для ручного вмешательства.

В таблице 9 описана сводка операций Redis по компонентам системы.

Таблица 9 – Матрица операций Redis по компонентам системы

Компонент	Операции Redis	Назначение
Lifecycle Manager	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление процессами
Proxy Engine	HGETALL, HMGET	Чтение данных заглушки
Rate Limiter	INCR, EXPIRE	Счётчик запросов
Settings Module	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление конфигурациями
Metrics Exporter	SMEMBERS, HGETALL (Pipeline)	Сбор метрик

Для Metrics Exporter применяется конвейеризация команд Redis (Pipeline): сначала читается список ключей из реестра, затем все операции чтения отправляются единым пакетом, снижая суммарные сетевые задержки с $O(n)$ до фактически $O(1)$ по числу циклов «запрос-ответ».

2.3 Разработка макетов приложения

Целевая аудитория системы – инженеры тестирования, одновременно отслеживающие состояние десятков заглушек. Это определяет два ключевых

требования: минимальную утомляемость при длительной работе и мгновенную читаемость статусов без анализа текстовых подписей.

Выбрана тёмная цветовая схема (dark theme) как стандарт для инструментов класса DevOps и Operations. Статусы объектов кодируются цветом однозначно: зелёный (RUNNING), жёлтый (REGISTERED / STOPPED), красный (ERROR / UNAVAILABLE). Акцентный цвет – синий #4F8EF7 с коэффициентом контрастности выше 4,5:1 (соответствует WCAG 2.1 уровня AA [49]). Основной шрифт – Inter (входит в базовую поставку большинства дистрибутивов Linux, не требует внешних CDN), моноширинный – JetBrains Mono для технических идентификаторов. Компоновка страниц: постоянная боковая панель навигации шириной 220 px и основная рабочая область. Сводная характеристика визуальной системы приведена в таблице 10.

Таблица 10 – Визуальная система интерфейса

Элемент	Значение	Назначение
Фон контента	#121218	Основной фон
Фон боковой панели	#1A1A24	Навигация
Акцентный цвет	#4F8EF7	Кнопки, ссылки
Статус RUNNING	#22C55E	Процесс запущен
Статус REGISTERED / STOPPED	#EAB308	Зарегистрирован, не запущен
Статус ERROR / UNAVAILABLE	#EF4444	Ошибка/недоступен
Основной текст	#E2E8F0	Основной контент
Вторичный текст	#94A3B8	Метки, подсказки
Основной шрифт	Inter	Текст
Моноширинный шрифт	JetBrains Mono	Код, пути, аргументы

Все экранные формы разделяют единый шаблон-«обёртку»: постоянная боковая панель с тремя разделами (Панель управления, Настройки, Журналы) и верхняя строка с заголовком текущего раздела и индикатором подключения к Redis. Макет общего каркаса приложения представлен на рисунке Б.1.

Панель управления (рисунок Б.2) является основным рабочим экраном. Верхняя часть содержит четыре сводные карточки: всего заглушек, запущено, ошибок, хостов доступно. Таблица заглушек включает столбцы: имя фай-

ла (моноширинный шрифт), целевой хост, статусный бейдж, порт, тумблер Rate Limiter и кнопки действий. Кнопка «Зарегистрировать заглушку» открывает модальное окно.

Модальное окно регистрации заглушки (рисунок Б.3) открывается поверх панели управления без перехода на отдельную страницу, сохраняя контекст. Форма содержит: зону перетаскивания файла (.jar/.war), выпадающий список целевого хоста, поле JVM-аргументов (моноширинный шрифт), числовое поле лимита Rate Limiter и флажок немедленного запуска после регистрации.

Страница настроек (рисунок Б.4) организована в виде двух вкладок: «Хосты» и «Учётные записи». Вкладка «Хосты» отображает таблицу целевых серверов с индикатором SSH-доступности и временем последней проверки. Пароли учётных записей скрыты строкой «.....». Кнопка «Добавить хост» открывает модальную форму (рисунок Б.5) с полями реестра.

Страница журналов (рисунок Б.6) предназначена для просмотра stdout/stderr Java-процессов. Выбор заглушки – через выпадающий список. Основную часть экрана занимает терминальный блок: строки с ERROR и WARN выделяются соответственно красным и жёлтым фоном. Панель управления содержит тумблер автообновления, кнопку обновления и очистки буфера; строка состояния показывает PID процесса и назначенный порт.

2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.

Все методы класса LifecycleManager реализованы как корутины Python (async/await), что позволяет событийному циклу asyncio [8] переключаться между операциями ввода-вывода без блокировки вычислительного потока. Операции управления ветвятся по типу целевого хоста: компонент Host Resolver предоставляет функцию `is_local(hostname)`, кэшированную через `functools.lru_cache`. Для локального хоста применяются встроенные средства Python и системные вызовы ОС; для удалённого – пул постоянных SSH-

соединений (SSHPool) на базе `asyncssh`. Ветвление инкапсулировано внутри `LifecycleManager` и прозрачно для API-слоя.

При запуске на локальном хосте поиск свободного TCP-порта выполняется в `asyncio.to_thread` (блокирующий `socket.bind` вынесен в пул потоков). JVM-аргументы из Redis разбираются функцией `shlex.split`, которая корректно обрабатывает кавычки и экранирование, — это позволяет задавать аргументы со встроенными пробелами: `”-Dmy.prop=value with spaces”`. Процесс запускается через `asyncio.create_subprocess_exec` с параметром `start_new_session=True`, обеспечивающим независимость Java-процесса от группы процессов управляющего приложения. Реализация приведена в листинге A.1.

При запуске на удалённом хосте поиск свободного порта производится через SSH-команду `ss -tln`. Токены JVM-аргументов дополнительно экранируются `shlex.quote` перед включением в строку команды, что предотвращает инъекцию оболочки. Конструкция `nohup & echo $!` запускает процесс в фоновом режиме и возвращает его PID в `stdout`. Формирование команды приведено в листинге A.2.

Остановка реализует двухэтапный протокол: сначала отправляется `SIGTERM`, затем в течение 5 секунд (50 итераций `asyncio.sleep(0.1)`) проверяется живость процесса; при неуспехе — `SIGKILL`. Ожидание через `asyncio.sleep` не блокирует событийный цикл, позволяя серверу обрабатывать другие запросы. Проверка живости ветвится аналогично запуску: локально — `os.kill(pid, 0)` в `asyncio.to_thread`, удалённо — `kill -0 {pid}` через SSH. Реализация обоих методов приведена в листинге A.3.

SSHPool хранит по одному постоянному соединению на хост, защищённому `asyncio.Lock` от гонок при переподключении. Перед каждым использованием соединение проверяется командой `true`; при неуспехе — пересоздаётся прозрачно для вызывающего кода. Передача артефактов реализована через `asyncssh.start_sftp_client` поверх существующего соединения, что исключает дополнительные SSH-сессии.

Описанная инфраструктура соединений задействуется в том числе при инициализации приложения. При старте метод `restore_state` обходит реестр заглушек и для каждой со статусом `RUNNING` проверяет фактическое наличие процесса на хосте. При подтверждении восстанавливается HTTP-клиент прокси-движка. При отсутствии процесса (или при повреждённых полях `pid/port`) система автоматически инициирует перезапуск: вызывается `_start_mock_from_data`, запись в Redis обновляется новыми `pid`, `port` и `started_at`. При неудаче ошибка фиксируется в журнале. Реализована стратегия «наилучшего усилия» (`best-effort restart`), минимизирующая ручное вмешательство оператора.

2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting

Модуль реализует прозрачный обратный прокси: входящий запрос к заглушке перенаправляется к её Java-процессу, ответ возвращается клиенту без изменений. Прокси-роутер подключён к FastAPI последним – после всех API-маршрутов и маршрутов веб-интерфейса, что исключает конфликты при совпадении имён заглушек с системными точками подключения.

Для каждой заглушки поддерживается отдельный постоянный `AsyncClient` [43] с пулом соединений (до 1000 одновременных, до 200 `keep-alive` с TTL 60 с). Класс `ProxyClientRegistry` управляет их созданием и закрытием; создание защищено `asyncio.Lock` против конкурентных запросов к новой заглушке; параметр `follow_redirects=False` гарантирует прозрачную передачу редиректов клиенту. Реализация приведена в листинге А.4.

Опираясь на реестр клиентов `ProxyClientRegistry`, прокси-роутер регистрирует два маршрута: `/ {mock_name}` и `/ {mock_name} / {path:path}` – оба делегируются единой функции `_handle_proxy`. Конвейер обслуживания включает следующие этапы:

1. Выполнение `HMGET` пяти полей заглушки из Redis.
2. Проверка статуса (`RUNNING`, иначе HTTP 503).

3. Чтение глобальных параметров (settings:global).
4. Проверка Rate Limiter (при превышении – HTTP 429).
5. Инкремент счётчика успешных запросов.
6. Получение клиента из ProxyClientRegistry.
7. Чтение тела запроса (кроме GET/HEAD/OPTIONS).
8. Вызов proxy_request и возврат ответа.
9. Сохранение метрик задержки в Redis.

Прозрачность обеспечивается передачей метода, тела и параметров запроса без изменений; заголовки пересылаются полностью, кроме Host и Content-Length, которые формируются заново. Транспортные ошибки унифицированы: TimeoutException – HTTP 504, RequestError – HTTP 502 [50]. Промежуточные этапы фиксируются в словаре timeline с точностью 0.001 мс для разделения задержки Redis, Rate Limiter и целевого хоста в Prometheus-метриках.

Включение и отключение Rate Limiter производится через PATCH-запрос в реальном времени без остановки заглушки: обновляется поле rate_limit_enabled в Hash-записи Redis, новое значение вступает в силу с первого же следующего запроса. Порог rate_limit изменяется аналогично – с начала следующего временного окна. Методы API приведены в таблице 11.

Таблица 11 – API-методы управления Rate Limiter

Метод	Путь	Описание
PATCH	/api/v1/mocks/{filename}/rate-limit	Включить или отключить Rate Limiter (тело: {"enabled": true/false})
PATCH	/api/v1/mocks/{filename}/config	Обновить порог лимита (rate_limit) и/или JVM-аргументы

2.6 Проектирование и программная реализация пользовательского интерфейса

Веб-интерфейс реализован по технологии Server-Side Rendering с применением шаблонизатора Jinja2 [41]. При каждом обращении к странице сервер формирует полный HTML-документ с актуальными данными из Redis и

передаёт браузеру в готовом виде. Подход устраняет зависимость от внешних JavaScript-фреймворков и CDN-ресурсов и обеспечивает корректную отрисовку страницы. Динамическое обновление таблиц и журналов реализовано стандартным JavaScript без сторонних библиотек: страница периодически опрашивает REST API и точно обновляет DOM (Document Object Model). Весь клиентский код сосредоточен в одном файле `web/static/js/app.js`, стили – в `web/static/css/style.css`.

Все страницы наследуют базовый шаблон `base.html`: повторяющаяся разметка (боковая панель, заголовочная полоса, подключение стилей и сценариев) определяется единожды; дочерние шаблоны переопределяют только блоки переменного содержания через `{% extends %}` / `{% block %}`. Состав шаблонного пакета приведён в таблице 12.

Таблица 12 – Шаблоны веб-интерфейса

Файл	Маршрут	Назначение
<code>base.html</code>	–	Общая оболочка: боковая навигация, заголовочная полоса, индикатор Redis
<code>dashboard.html</code>	GET /	Таблица заглушек, сводные карточки, диалог регистрации
<code>settings.html</code>	GET /settings	Управление хостами и учётными записями SSH
<code>logs.html</code>	GET /logs	Блок вывода журналов Java-процессов

Маршруты веб-интерфейса обслуживаются FastAPI-роутером `web.routes`, подключённым первым – до API-маршрутов и прокси-роутера. Каждый маршрут выполняет минимальную выборку данных из Redis через конвейер команд (`transaction=False`): команды `HGETALL` для всех заглушек отправляются одним пакетом и считываются единственным вызовом `pipe.execute()`, что исключает пропорциональный рост числа сетевых обращений. Реализация маршрута панели управления приведена в листинге А.6.

Базовый шаблон реализует двухколоночный макет: фиксированная боковая панель шириной 220 px (`.sidebar`) и основная рабочая область (`.main-area`). Активный пункт навигации определяется переменной контекста `active_page`, передаваемой из каждого маршрута: шаблон добавляет CSS-

класс `is-active` к соответствующей ссылке через условие Jinja2 – без клиентского кода.

В правом верхнем углу заголовочной полосы отображается индикатор состояния Redis: зелёная точка и «Redis: подключён» при успешном ответе на PING, красная и «Redis: отключён» при сбое. Состояние передаётся в переменную контекста `redis_connected` и формируется через условный блок `{% if %}` с атрибутом `role="status"` для программ экранного доступа.

Верхняя часть панели содержит четыре сводные карточки (всего заглушек, RUNNING, ERROR, хостов доступно) для быстрой оценки состояния инфраструктуры. Таблица заглушек включает шесть столбцов: имя файла, целевой хост, статусный бейдж, TCP-порт, тумблер Rate Limiter и кнопки управления. Бейджи реализованы через CSS-классы `badge--running/error/registered/stopped`; кнопки «Старт» и «Стоп» взаимоисключающие – шаблон отображает одну из них по текущему статусу.

Тумблер Rate Limiter реализован как `<input type="checkbox">` со специализированной CSS-разметкой `.toggle`; изменение состояния немедленно отправляет PATCH-запрос к `/api/v1/mocks/{filename}/rate-limit` без перезагрузки страницы. Таблица обновляется каждые 5 секунд через `setInterval` (интервал задаётся атрибутом `data-poll-interval-ms="5000"`): параллельные запросы к `/api/v1/mocks` и `/api/v1/hosts` выполняются через `Promise.all`, полученные данные точно обновляют DOM без полного перестроения таблицы. Реализованная главная панель представлена на рисунке В.1.

Регистрация выполняется через диалоговое окно поверх панели управления без перехода на отдельную страницу, сохраняя контекст. Форма содержит пять полей, приведённых в таблице 13.

Таблица 13 – Поля формы регистрации заглушки

Поле	Тип	Описание
1	2	3
file	Область перетаскивания	.jar/.war до 512 МБ; поддерживает drag-and-drop
hostname	Раскрывающийся список	Целевой хост из реестра; список формируется сервером при генерации

Продолжение таблицы 13

1	2	3
jvm_args	Многострочное поле	JVM-аргументы; моноширинный шрифт
rate_limit	Числовое поле	Лимит запросов в секунду; 0 – без ограничений
auto_start	Флажок	Запустить заглушку после регистрации

Форма отправляется как multipart/form-data на POST /api/v1/mocks. Сервер отвечает кодом 202 (Accepted): SFTP-передача файла выполняется в фоне через BackgroundTasks FastAPI, что исключает зависание браузера при передаче крупных артефактов. После ответа 202 клиентский код закрывает диалог и через 1 секунду инициирует принудительное обновление таблицы. Область перетаскивания реализована на стандартных событиях dragover/dragleave/drop: при наведении рамка подсвечивается акцентным цветом, при сбросе отображается имя файла. Реализованное диалоговое окно представлено на рисунке В.2.

Страница организована в две вкладки («Хосты» и «Учётные записи»), переключаемые JavaScript без перезагрузки и реализованные по спецификации ARIA (role="tablist", role="tab", role="tabpanel"): при выборе вкладки устанавливаются aria-selected и CSS-класс is-active.

Вкладка «Хосты» отображает таблицу целевых серверов: имя хоста, порт SSH, рабочую директорию, статус SSH-соединения (AVAILABLE/UNAVAILABLE/UNKNOWN), дату последней проверки и описание. Все параметры хоста хранятся в атрибутах data-* строки таблицы: при открытии формы редактирования клиентский код считывает их и заполняет поля без дополнительного запроса к API.

Вкладка «Учётные записи» отображает список учётных записей Linux. Пароли скрыты строкой (aria-hidden="true" для программ экранного доступа) – это предотвращает случайную утечку при демонстрации системы. Пароль шифруется на сервере алгоритмом AES-128 (Fernet) перед записью в Redis. Реализованные страницы настроек представлены на рисунках В.3 и В.4.

Страница предназначена для просмотра stdout/stderr Java-процессов в режиме, близком к реальному времени. Выбор заглушки – через раскрывающийся список; при смене выбора клиент последовательно запрашивает GET /api/v1/mocks/{filename} (текущие PID и порт) и GET /api/v1/logs/{filename} (строки журнала). Строка состояния над блоком вывода отображает имя файла, PID и порт процесса.

Блок вывода (.terminal) реализован как <div> с тёмным фоном #0D0D0D и шрифтом JetBrains Mono. Функция formatLogLine выделяет временную метку регулярным выражением (ISO 8601 или в квадратных скобках) и окрашивает её в #64748B; строки с WARN получают жёлтый фон и класс terminal__line--warn, с ERROR – красный фон и класс terminal__line--error. Автообновление реализовано через setInterval с интервалом 2 секунды, управляемым флажком «Авто». Кнопка «Очистить» вызывает DELETE /api/v1/logs/{filename}, что сбрасывает буфер в Redis-ключе logs:{filename}. Реализованная страница представлена на рисунке В.5.

Клиентский код не использует библиотек привязки событий. Обработчики click и change устанавливаются на корневые элементы страниц; тип действия определяется атрибутом data-action ближайшего подходящего элемента. Динамически добавляемые строки таблицы при периодическом обновлении немедленно обрабатываются существующими обработчиками без дополнительной регистрации.

Все обращения к REST API выполняются через fetch() с однотипной обработкой ошибок: поле detail из JSON-ответа отображается в диалоге предупреждения. Кнопки на время запроса переводятся в неактивное состояние через setButtonLoading: разметка сохраняется в dataset.origHtml и заменяется индикатором загрузки, атрибут disabled блокирует повторное нажатие.

Все цветовые и типографические константы определены как CSS-переменные в секции :root файла style.css. Оба шрифта – Inter и JetBrains Mono – подключаются через @font-face из локальной директории web/static/fonts/ без внешних CDN, что соответствует требованию техноло-

гической независимости (раздел 1.3). Ключевые переменные оформления приведены в таблице 14.

Таблица 14 – Переменные системы визуального оформления интерфейса

Переменная	Значение	Назначение
--bg-main	#121218	Основной фон рабочей области
--bg-sidebar	#1A1A24	Фон боковой панели навигации
--bg-card	#1E1E2E	Фон карточек и диалоговых окон
--accent	#4F8EF7	Акцентный цвет кнопок и активных элементов
--text-primary	#E2E8F0	Основной текст
--text-secondary	#94A3B8	Метки, подсказки
--status-running	#22C55E	Статус RUNNING
--status-registered	#EAB308	Статусы REGISTERED / STOPPED
--status-error	#EF4444	Статус ERROR / UNAVAILABLE
--bg-terminal	#0D0D0D	Фон блока вывода журналов

2.7 Реализация механизма экспорта метрик мониторинга

Механизм мониторинга построен по pull-модели Prometheus [45]: внешняя система периодически опрашивает маршрут GET /metrics, а управляющее приложение возвращает актуальный снимок показателей в формате Prometheus exposition format [46]. Метрические данные хранятся в Redis и обновляются в процессе обслуживания каждого проксированного запроса: функция collect_metrics при обращении к /metrics не вычисляет значения, а лишь считывает уже агрегированные данные за одну пакетную операцию.

Реализация разделена на две части: функция накопления record_proxy_observation, вызываемая в конце каждого проксированного запроса, и функция экспорта collect_metrics, вызываемая по каждому запросу к /metrics. Система экспортирует двенадцать метрических серий двух типов по спецификации Prometheus: счётчики (counter, монотонно возрастают) и измерители (gauge, текущее мгновенное значение). Полный состав метрик приведён в таблице 15.

Таблица 15 – Экспортируемые метрики системы

Имя метрики	Тип	Описание
mockcontrol_mock_total	gauge	Число зарегистрированных заглушек
mockcontrol_mock_running	gauge	Число заглушек в состоянии RUNNING
mockcontrol_mock_errors	gauge	Число заглушек в состоянии ERROR
mockcontrol_hosts_available	gauge	Число хостов в состоянии AVAILABLE
mockcontrol_proxy_requests_total	counter	Успешно проксированных запросов (метка mock)
mockcontrol_rate_limit_rejected_total	counter	Отклонённых запросов по Rate Limiter (метка mock)
mockcontrol_proxy_stage_latency_ms_sum	counter	Накопленная сумма задержек по этапам, мс (метки mock, stage)
mockcontrol_proxy_stage_latency_ms_count	counter	Число наблюдений задержки по этапам (метки mock, stage)
mockcontrol_proxy_stage_latency_ms_avg	gauge	Среднее время этапа, мс (метки mock, stage)
mockcontrol_proxy_stage_latency_ms_last	gauge	Последнее измеренное время этапа, мс (метки mock, stage)
mockcontrol_proxy_outcome_total	counter	Исходы проксирования по типу (метки mock, outcome)
mockcontrol_proxy_upstream_status_total	counter	Ответы заглушки по HTTP-коду (метки mock, status)

Все метрики с меткой `mock` дополнительно агрегируются по значению `mock="__all__"`, что позволяет получать суммарные показатели по всем заглушкам единым Prometheus-запросом. Метка `stage` принимает значения этапов функции `_handle_proxy`: `redis_mock_hmget_ms`, `rate_limiter_check_ms`, `httpx_proxy_request_ms`, `proxy_total_ms` и др. Метка `outcome` принимает семь значений: `ok`, `timeout`, `request_error`, `not_found`, `not_running`, `invalid_target`, `rate_limited`.

По завершении каждого запроса функция `record_proxy_observation` обновляет показатели Redis через единственный конвейер (`transaction=False`): для каждого этапа выполняются `HINCRBYFLOAT` (сумма задержек), `HINCRBY` (счётчик наблюдений) и `HSET` (последнее значение) – как для ключа конкретной заглушки, так и для агрегирующего ключа `__all__`. Параметр `transaction=False` использован намеренно: частичная видимость проме-

жуточных значений допустима, поскольку Prometheus считывает метрики с интервалом в десятки секунд.

Функция `collect_metrics` собирает все данные за два конвейерных прохода. В первом запрашиваются реестры `mocks:registry` и `hosts:registry`; во втором – единым пакетом: `HGETALL` каждой заглушки, `HGET` статуса каждого хоста и семь метрических ключей на заглушку плюс пять агрегирующих ключей `__all__`. Число сетевых транзакций фиксировано: два цикла «запрос-ответ» вне зависимости от числа заглушек. Реализация приведена в листинге А.7.

После получения пакета функция разбирает результаты по позиционному индексу: первые `n_m` элементов – Hash-записи заглушек, следующие `n_h` – статусы хостов, оставшиеся – метрические ключи в порядке добавления в конвейер. Ответ формируется по спецификации Prometheus exposition format 0.0.4 [46]: каждая серия открывается строками `# HELP` и `# TYPE`, после чего следуют строки значений. Значения меток экранируются функцией `_escape_label_value` (обратный слеш, перевод строки, двойные кавычки), что обеспечивает корректный разбор при наличии специальных символов в именах файлов заглушек. Фрагмент ответа приведён в листинге А.8.

2.8 Выводы

Во второй главе выпускной квалификационной работы выполнены проектирование архитектуры, разработка структур данных и программная реализация всех ключевых компонентов разработанного программного комплекса.

На основании сравнительного анализа архитектурных стилей обоснован выбор монолитной архитектуры с явно выраженной внутренней модульной организацией. Показано, что данное решение оптимально соответствует требованиям проекта: исключает накладные расходы на межсервисное взаимодействие, устраняет зависимость от внешних систем оркестрации контейнеров и обеспечивает простоту развёртывания в закрытом корпоративном

контуре. Разработана гибкая топология развёртывания, в рамках которой единый управляющий процесс координирует как локальные, так и удалённые целевые хосты по единому алгоритму через механизмы `asyncssh/SFTP` без установки дополнительного программного обеспечения на целевые серверы.

Архитектура системы формализована в нотации `C4 Model` на трёх уровнях детализации: контекстном, контейнерном и компонентном. Выделены и описаны шесть программных компонентов – `Lifecycle Manager`, `Proxy Engine`, `Rate Limiter`, `Settings Module`, `Crypto Service` и `Metrics Exporter`.

Спроектированы структуры данных `Redis` для хранения конфигураций заглушек, хостов и учётных записей в виде `Hash`-записей с обеспечением доступа за одну операцию `HGETALL`. Реализован алгоритм ограничения интенсивности запросов `Fixed Window Counter` на основе атомарной операции `INCR` с автоматической очисткой устаревших счётчиков через механизм `TTL`. Разработана стратегия восстановления состояния после перезапуска системы: проверка фактической живости каждого процесса с последующим обновлением `Redis`-записей до согласованного состояния.

Реализованы асинхронные механизмы управления жизненным циклом `Java`-процессов с применением `asyncio.create_subprocess_exec` и `asyncio.to_thread` для исключения блокирования событийного цикла на операциях ввода-вывода. Разработан механизм безопасного разбора пользовательских `JVM`-аргументов на основе лексического анализатора `shlex`, обеспечивающий защиту от инъекции команд оболочки при запуске процессов на удалённых хостах. Реализован пул постоянных `SSH`-соединений, снижающий задержку операций управления на удалённых хостах за счёт исключения повторного согласования ключей при каждой операции.

Разработан модуль проксирования запросов на основе асинхронного `HTTP`-клиента `httpx` с индивидуальным пулом соединений для каждой заглушки. Реализована прозрачная маршрутизация входящего трафика с сохранением метода, заголовков и тела запроса. Обоснована применимость алгоритма `Fixed Window Counter` для задач ограничения интенсивности запросов

в среде нагрузочного тестирования с учётом его операционной стоимости и допустимой погрешности на границах временных окон [51].

Спроектирован и реализован веб-интерфейс на основе технологии формирования страниц на стороне сервера с применением шаблонизатора Jinja2. Интерфейс не требует внешних зависимостей: оба шрифта и весь клиентский код подключаются из локальных файлов без обращения к внешним источникам. Реализован механизм делегирования событий, обеспечивающий корректную обработку динамически добавляемых элементов таблицы без дополнительной регистрации обработчиков.

Реализован механизм экспорта метрик мониторинга в формате Prometheus exposition format. Разработана двухпроходная схема конвейерной сборки данных из Redis, обеспечивающая формирование ответа за фиксированное число сетевых транзакций вне зависимости от количества зарегистрированных заглушек. Определены двенадцать метрических серий с детализацией задержек по этапам конвейера обработки запроса, что обеспечивает возможность инструментальной диагностики узких мест инфраструктуры в ходе нагрузочного тестирования.

Полученные результаты образуют завершённую программную реализацию системы, готовую к развёртыванию в операционной среде и верификации в рамках тестирования, описанного в третьей главе.

3 ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА

3.1 Описание процесса развёртывания системы в среде операционной системы Astra Linux

Развёртывание программного комплекса выполняется на сервере под управлением операционной системы Astra Linux [3,25,35]. Минимальные требования к аппаратной конфигурации управляющего сервера приведены в таблице 16.

Таблица 16 – Минимальные требования к аппаратной конфигурации

Параметр	Значение
Процессор	8 vCPU
Оперативная память	8 ГБ
Дисковое пространство	Не менее 10 ГБ
Сетевой интерфейс	Ethernet; SSH-доступность целевых хостов
Операционная система	Astra Linux Special Edition / Common Edition

На управляющем сервере должны быть установлены интерпретатор Python версии 3.10 и выше, а также хранилище Redis версии 5.x с включённым режимом персистентности AOF [23].

На целевых хостах, где исполняются имитационные сервисы, требуется OpenJDK 17 или более поздней версии [36]. Установка какого-либо дополнительного программного обеспечения агента на целевые хосты не требуется: взаимодействие с удалёнными узлами осуществляется исключительно через протокол SSH.

На первом этапе средствами штатного пакетного менеджера Astra Linux устанавливаются Python 3, менеджер пакетов `pip`, модуль виртуальных окружений `venv` и сервис Redis. Redis включается в автозагрузку и запускается.

На втором этапе создаётся изолированное виртуальное окружение Python, что исключает конфликты с системными пакетами. После его активации выполняется установка зависимостей проекта из файла `requirements.txt`. Состав зависимостей включает: FastAPI [14], Uvicorn [18], `asyncssh` (SSH/SFTP-клиент), `httpx` [43], `pydantic-settings` [17], `cryptography` [44],

redis-py, а также вспомогательные библиотеки aiofiles, Jinja2 [41] и python-multipart.

После установки зависимостей настраиваются параметры запуска приложения. Все параметры вынесены в файл .env, создаваемый на основе поставляемого шаблона .env.example. Параметры считываются модулем pydantic_settings через класс Settings при инициализации приложения. Основные параметры приведены в таблице 17.

Таблица 17 – Параметры конфигурационного файла .env

Параметр	По умолчанию	Назначение
REDIS_URL	redis://localhost:6379/0	URL подключения к Redis
FERNET_KEY_PATH	/etc/mockcontrol/fernet.key	Путь к файлу ключа Fernet
TEMP_UPLOAD_DIR	/tmp/mockcontrol	Директория временных загрузок
DEFAULT_RATE_LIMIT_WINDOW	1	Размер окна Rate Limiter (сек)
DEFAULT_HOST_CHECK_INTERVAL	30	Интервал проверки хостов (сек)
DEFAULT_PROXY_TIMEOUT	10	Время ожидания ответа заглушки (сек)
DEFAULT_LOG_RETENTION_LINES	1000	Максимум строк лога в буфере

Для защиты паролей SSH-учётных записей применяется симметричное шифрование по алгоритму Fernet [44]. Ключ генерируется однократно при первоначальном развёртывании и сохраняется по пути, заданному в параметре FERNET_KEY_PATH. Директория создаётся с правами доступа, ограничивающими чтение файла ключа только для учётной записи службы.

В конфигурационном файле Redis активируется режим персистентности AOF директивой appendonly yes. Это гарантирует восстановление всех записанных операций при аварийном перезапуске хранилища [22,23]. Дополнительно рекомендуется привязать Redis к адресу 127.0.0.1 при совместном размещении с приложением, а политику вытеснения задать равной noeviction, поскольку произвольное удаление ключей конфигурации недопустимо.

Для обеспечения автоматического запуска при загрузке операционной системы и перезапуска при аварийном завершении приложение регистрируется в качестве системной службы `systemd`. Конфигурационный файл службы размещается в директории `/etc/systemd/system/`. Полный текст `unit`-файла приведен в листинге А.9.

Директива `After=network.target redis.service` обеспечивает активацию службы только после готовности сетевого стека и хранилища Redis. Параметр `ExecStart` задаёт команду запуска: интерпретатор Python из виртуального окружения с аргументом `main.py`. Директивы `Restart=on-failure` и `RestartSec=5` настраивают автоматический перезапуск при ненулевом коде завершения с паузой в 5 секунд. Параметр `User` определяет учётную запись, от имени которой выполняется процесс (не `root`).

После создания файла выполняется перезагрузка демона (`systemctl daemon-reload`), регистрация в автозагрузке (`systemctl enable`) и первоначальный запуск службы. Текущее состояние и журнал службы доступны через стандартные команды `systemctl status` и `journalctl`.

При запуске зарегистрированной службы приложение выполняет последовательную инициализацию всех компонентов. Приложение реализует ASGI-интерфейс [16] и запускается посредством сервера Uvicorn [18]. В процессе инициализации выполняется жизненный цикл `lifespan`: устанавливается соединение с Redis, создаются экземпляры внутренних сервисов, вызывается процедура восстановления состояния (`restore_state`), которая проверяет фактическую доступность заглушек, числившихся активными в момент последней остановки. Затем при наличии соответствующих флагов в глобальных настройках запускаются фоновые задачи – планировщик проверки доступности хостов и сборщик журналов процессов.

После успешного запуска веб-интерфейс доступен по адресу `http://<IP>:8000`. Корректность соединения с Redis подтверждается индикатором в боковой навигации интерфейса (рисунок В.1). API-метод метрик в формате Prometheus exposition format [46] доступен по пути

/metrics и может использоваться для интеграции с внешней системой мониторинга [45,47].

После запуска управляющего сервера в разделе «Настройки» веб-интерфейса регистрируются целевые хосты и SSH-учётные записи. Для каждого хоста указываются: сетевой адрес, порт SSH, рабочая директория для размещения артефактов и учётная запись. При локальном развёртывании (адрес 127.0.0.1) SSH-соединение не устанавливается – операции выполняются напрямую через системные вызовы, что определяется компонентом HostResolver. Статус доступности хоста обновляется в рамках очередного цикла планировщика HealthChecker и отображается в интерфейсе (OK / ERROR / UNKNOWN).

3.2 Тестирование механизмов автоматического восстановления состояний и перезапуска сервисов

Данный раздел посвящён экспериментальной проверке двух взаимосвязанных механизмов системы: восстановления состояния реестра заглушек при перезапуске управляющего приложения и непрерывного контроля доступности запущенных сервисов фоновым планировщиком HealthChecker. Оба механизма обеспечивают отказоустойчивость инфраструктуры имитационных сервисов без ручного вмешательства оператора.

Для подтверждения корректности реализации были проведены следующие сценарии испытаний:

1. «Холодный» перезапуск управляющего приложения при работающих заглушках (штатная ситуация).
2. Перезапуск управляющего приложения при принудительно завершённых процессах заглушек (аварийная ситуация – автоматический перезапуск).
3. Перезапуск управляющего приложения после аварийного завершения Redis с последующим восстановлением данных из AOF-журнала.

4. Обнаружение HealthChecker'ом факта аварийного завершения заглушки в штатном режиме работы (без перезапуска приложения).

5. Проверка корректности смены статуса заглушки при устранении причины сбоя (автоматическое возвращение из ERROR в RUNNING).

Тестирование проводилось на развёрнутом стенде под управлением ОС Astra Linux. Конфигурация стенда: управляющий сервер 8/8, на котором одновременно располагались управляющее приложение, Redis и целевые имитационные сервисы (локальный режим развёртывания).

Сценарий 1. Перезапуск приложения при работающих заглушках – проверка корректности восстановления состояния при наличии активных процессов.

1. Подготовка: в реестр системы зарегистрированы 2 имитационных сервиса, все переведены в состояние RUNNING.

2. Действие: управляющее приложение остановлено командой `systemctl stop mockcontrol` и запущено повторно командой `systemctl start mockcontrol`.

3. Ожидаемый результат: процедура `restore_state` при инициализации обходит реестр заглушек, проверяет фактическое наличие каждого процесса по сохранённому в Redis полю `pid`, подтверждает его активность и восстанавливает HTTP-клиент прокси-движка; статусы в Redis и отображение в интерфейсе остаются RUNNING.

4. Наблюдаемый результат: после перезапуска все заглушки отображаются в интерфейсе со статусом RUNNING; журнал службы (листинг А.10) фиксирует успешное завершение процедуры восстановления для каждого сервиса.

5. Вывод: механизм восстановления корректно сопоставляет сохранённые метаданные (PID, порт, хост) с фактическим состоянием операционной системы и не нарушает работу уже запущенных сервисов.

Сценарий 2. Перезапуск приложения при завершённых процессах заглушек – проверка стратегии автоматического перезапуска при обнаружении отсутствующих процессов.

1. Подготовка: 2 заглушки в состоянии RUNNING; до перезапуска управляющего приложения процессы заглушек принудительно завершены командой kill по их PID.

2. Действие: управляющее приложение перезапущено.

3. Ожидаемый результат: процедура restore_state обнаруживает отсутствие процессов (сигнал kill -0 возвращает ошибку либо системный вызов os.kill(pid, 0) выбрасывает исключение OSError); для каждой такой заглушки автоматически вызывается _start_mock_from_data – перезапуск с сохранёнными в Redis JVM-аргументами и путём к артефакту; Redis-запись обновляется новыми значениями pid, port и started_at.

4. Наблюдаемый результат: после завершения инициализации все заглушки отображаются в состоянии RUNNING; журнал службы (листинг A.11) содержит записи об обнаружении отсутствующих процессов и об успешном перезапуске каждого из них.

5. Вывод: стратегия наилучшего усилия (best-effort restart) обеспечивает автоматическое восстановление инфраструктуры заглушек без участия оператора при аварийном завершении Java-процессов в период простоя управляющего приложения.

Сценарий 3. Восстановление данных после аварийного завершения Redis – проверка надёжности персистентности при использовании AOF-журнала.

1. Подготовка: 2 заглушки в состоянии RUNNING.

2. Действие: процесс Redis принудительно завершён сигналом SIGKILL (имитация аварийного сбоя без штатного сохранения); Redis и управляющее приложение перезапущены.

3. Ожидаемый результат: Redis при запуске воспроизводит все операции из AOF-журнала (appendonly.aof), восстанавливая данные реестра заглу-

шек с потерей не более одной секунды операций (в соответствии с директивой `appendfsync everysec` [23]); управляющее приложение обнаруживает восстановленные данные и выполняет стандартную процедуру `restore_state`.

4. Наблюдаемый результат: конфигурации всех зарегистрированных заглушек, хостов и учётных записей восстановлены; заглушки в состоянии `RUNNING` перезапущены автоматически; данные интерфейса соответствуют состоянию до сбоя Redis; журнал Redis (листинг А.12) подтверждает успешное воспроизведение AOF-файла.

5. Вывод: режим AOF с политикой `appendfsync everysec` обеспечивает надёжную персистентность данных системы при аварийном завершении хранилища [22].

Сценарий 4. Обнаружение аварийного завершения заглушки планировщиком `HealthChecker` – проверка своевременного обновления статуса в рамках заданного интервала проверки.

1. Подготовка: заглушка работает в состоянии `RUNNING`; интервал проверки планировщика установлен в значение 30 секунд (параметр `DEFAULT_HOST_CHECK_INTERVAL` или настройка в интерфейсе).

2. Действие: Java-процесс заглушки принудительно завершён сигналом `SIGKILL` при работающем управляющем приложении.

3. Ожидаемый результат: в течение следующего цикла планировщика `HealthChecker` выполняются две независимые проверки – проверка живости процесса по PID (локальный вызов `os.kill(pid, 0)` или SSH-команда `kill -0 PID` для удалённого хоста) и HTTP-опрос корневого маршрута заглушки; при отрицательном результате PID-проверки статус заглушки немедленно обновляется до `ERROR` без ожидания нескольких неудачных HTTP-попыток; цветовой индикатор в интерфейсе меняется с зелёного (`RUNNING`) на красный (`ERROR`).

4. Наблюдаемый результат: не позднее чем через 30 секунд статус заглушки изменился на `ERROR`; интерфейс системы после обнаружения сбоя

приведён на рисунке 6; журнал службы (листинг А.13) зафиксировал предупреждение «process not found».

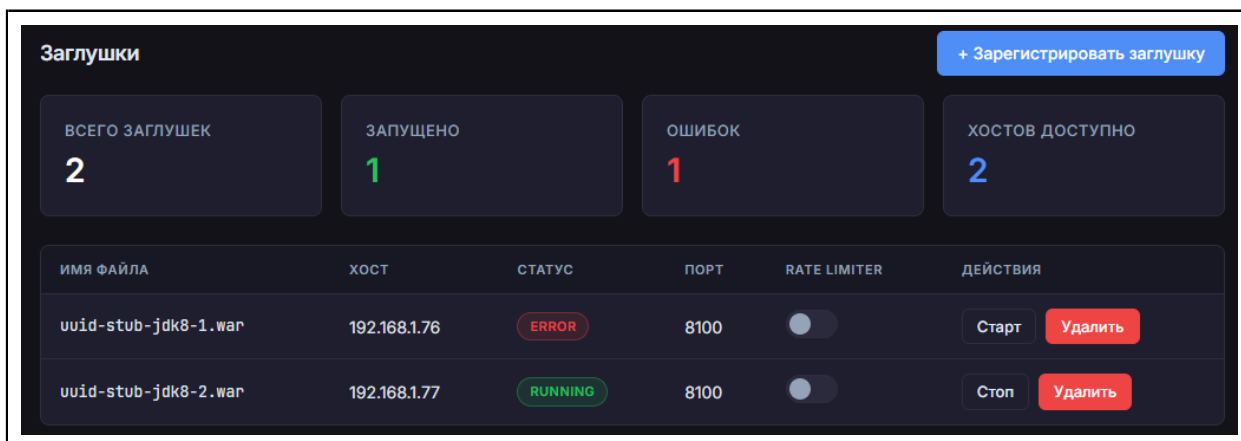


Рисунок 6 – Интерфейс панели управления: заглушка переведена в статус ERROR после обнаружения аварийного завершения процесса

5. Вывод: HealthChecker обеспечивает своевременное обнаружение аварийного завершения Java-процессов и актуализацию статуса в реестре в рамках заданного интервала проверки.

Сценарий 5. Автоматическое возвращение заглушки из состояния ERROR – проверка восстановления статуса без вмешательства оператора после устранения причины сбоя.

1. Подготовка: заглушка находится в состоянии ERROR (аварийное завершение процесса из сценария 4); оператор вручную запускает Java-процесс заглушки или использует кнопку «Старт» в интерфейсе для перезапуска.

2. Действие: заглушка запущена штатным образом; ожидается следующий цикл HealthChecker.

3. Ожидаемый результат: HealthChecker в следующем цикле проверки подтверждает, что PID-проверка успешна и HTTP-опрос корневого маршрута возвращает ответ (любой код HTTP считается признаком активности сервиса [43]); счётчик накопленных HTTP-отказов для данной заглушки сбрасывается; статус в Redis обновляется до RUNNING.

4. Наблюдаемый результат: не позднее чем через 30 секунд статус заглушки сменился с ERROR на RUNNING без дополнительных действий опе-

ратора; итоговое состояние интерфейса приведено на рисунке 7; запись о восстановлении зафиксирована в журнале службы (листинг А.14).

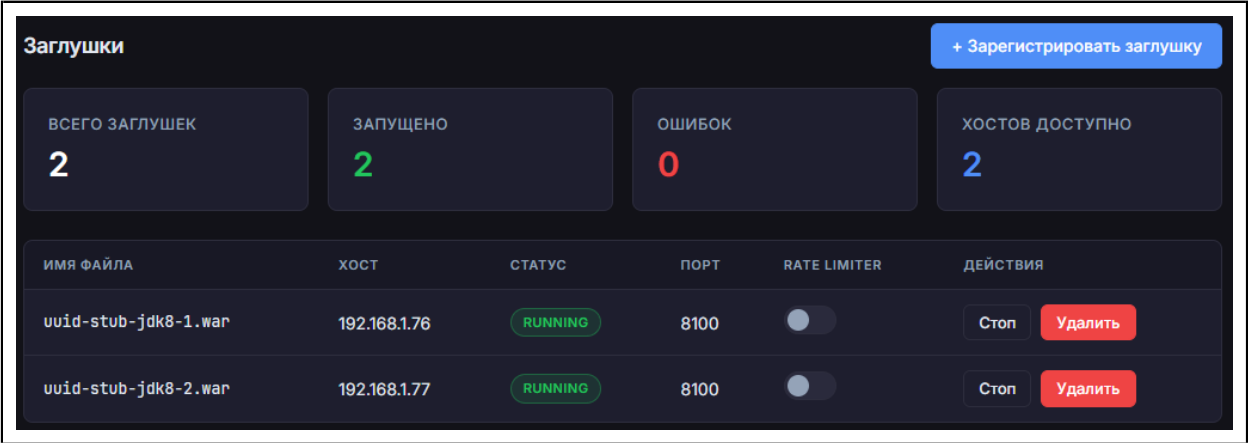


Рисунок 7 – Интерфейс панели управления:
заглушка вернулась в состояние RUNNING

5. Вывод: логика HealthChecker допускает проверку заглушек, находящихся в состоянии ERROR, что обеспечивает автоматическое возвращение сервиса в штатное состояние после устранения причины сбоя без перезапуска управляющего приложения.

Результаты всех проведённых сценариев сведены в таблицу 18.

Таблица 18 – Результаты испытаний механизмов восстановления

Сценарий	Результат	Примечание
Перезапуск при работающих процессах	Пройден	Статусы RUNNING сохранены
Перезапуск при завершённых процессах	Пройден	Автоматический перезапуск заглушек
Восстановление после аварии Redis	Пройден	Данные восстановлены из AOF
Обнаружение завершения HealthChecker'ом	Пройден	Статус ERROR за ≤ 30 сек
Автовозврат из ERROR после перезапуска	Пройден	Статус RUNNING за ≤ 30 сек

Совокупность проведённых испытаний подтверждает, что реализованные механизмы персистентности и восстановления обеспечивают устойчивость инфраструктуры имитационных сервисов к типичным эксплуатационным сбоям: перезапускам управляющего приложения, аварийному завершению Java-процессов и отказам хранилища данных.

3.3 Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой

Целью данного испытания является экспериментальная проверка двух ключевых характеристик разработанного программного комплекса: производительности прокси-модуля при одновременной обработке множества HTTP-запросов и корректности работы алгоритма Rate Limiting под нагрузкой. Оцениваются следующие показатели: сквозная задержка обработки одного запроса (end-to-end latency, метрика `proxy_total_ms`), задержка на каждом из внутренних этапов конвейера, пропускная способность прокси-модуля (запросов в секунду, RPS), а также доля запросов, отклонённых алгоритмом Rate Limiting при превышении установленного лимита.

Нагрузка генерируется инструментом k6 [52]. Наблюдение за поведением системы ведётся через точку подключения `/metrics`: после каждого нагрузочного этапа из него считываются агрегированные покомпонентные задержки. Среднее время этапа вычисляется как отношение накопленной суммы (`_sum`) к числу наблюдений (`_count`).

Встроенный инструментарий фиксирует три этапа конвейера: метрика `httpx_proxy_request_ms` отражает пересылку запроса к заглушке и получение ответа; метрика `httpx_pool_wait_ms` – ожидание свободного соединения в пуле `httpx`; метрика `proxy_total_ms` – полную сквозную задержку от получения запроса до возврата ответа. Разбиение на этапы позволяет локализовать источник задержки и отделить накладные расходы самого прокси от времени отклика целевого сервиса.

Параметры. Нагрузка: 10 RPS, 1 параллельное соединение, длительность – 60 секунд. Rate Limiting отключён. Данный сценарий служит базовой линией: при минимальной конкуренции за ресурсы фиксируется нижняя граница задержки, с которой сравниваются результаты последующих сценариев.

Наблюдаемые результаты. Значения по этапам приведены в таблице 19. Метрики прокси-модуля с разбивкой задержки по всем этапам конвейера представлены на рисунке 8.

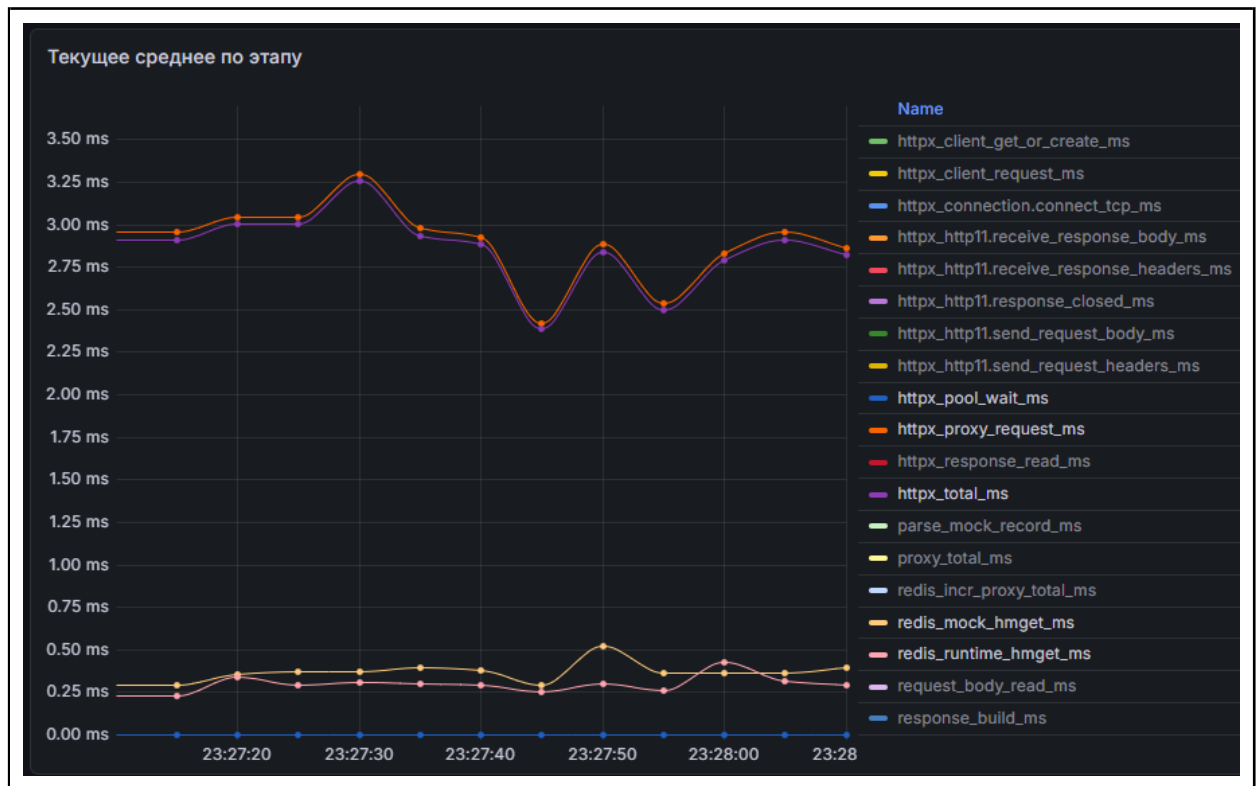


Рисунок 8 – Метрики прокси-модуля при базовой нагрузке 10 RPS: разбивка задержки по этапам конвейера

Таблица 19 – Задержки этапов конвейера при нагрузке 10 RPS

Этап	Среднее, мс
httpx_proxy_request_ms	2,90
httpx_pool_wait_ms	0,00
proxy_total_ms	3,90

Анализ результатов. При малой нагрузке (10 RPS, 1 соединение) прокси-модуль демонстрирует устойчиво низкую сквозную задержку: среднее значение proxy_total_ms составило 3,90 мс. Из них 2,90 мс приходится на этап httpx_proxy_request_ms – фактическую пересылку запроса к заглушке и получение ответа, – то есть на долю накладных расходов самого прокси-модуля (разбор записи из Redis, формирование ответа) приходится порядка 1 мс. Задержка ожидания в пуле соединений (httpx_pool_wait_ms) равна нулю: при единственном параллельном соединении конкуренции за пул не возникает. Полученные базовые значения характеризуют «идеальную» задержку.

ку системы и служат контрольной точкой для оценки деградации при нарастании нагрузки.

Параметры. Rate Limiting отключён. Нагрузка последовательно увеличивается: 125, 250, 500, 1000 RPS. Длительность каждого этапа – 120 секунд.

Наблюдаемые результаты. Зависимость сквозной задержки и задержки ожидания в пуле от интенсивности нагрузки приведена в таблице 20. Временные ряды метрик в ходе испытания представлены на рисунке 9.

Таблица 20 – Зависимость задержки от интенсивности нагрузки

Нагрузка, RPS	proxy_total_ms, мс	httpx_pool_wait_ms, мс	Факт. RPS
125	2,54	0,00	125
250	2,70	0,00	250
500	3,01	0,00	500
1000	10,52	0,00	1000

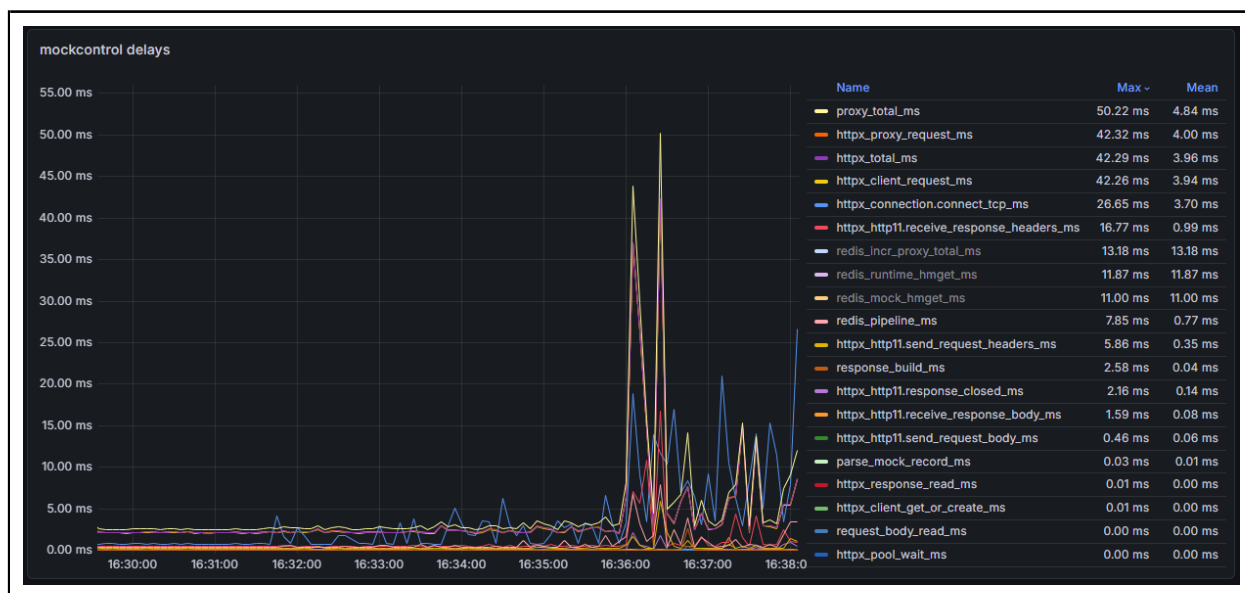


Рисунок 9 – Временные ряды метрик прокси-модуля в ходе нагрузочного испытания: нарастающая нагрузка 125–1000 RPS

Анализ результатов. Результаты испытания демонстрируют две отчётливые зоны поведения прокси-модуля. В диапазоне 125–500 RPS (зона линейного масштабирования) сквозная задержка `proxy_total_ms` возрастает незначительно – с 2,54 мс до 3,01 мс, то есть менее чем на 20 % при четырёхкратном росте нагрузки. Прирост объясняется увеличением конкуренции за планировщик `event-loop`'а асинхронного сервера `Uvicorn`, тогда как задержка ожидания соединений в пуле `httpx` остаётся нулевой на всём диапазоне:

пул успевает обслуживать входящие корутины без образования очереди ожидания. Таким образом, в данном диапазоне нагрузок прокси-модуль демонстрирует предсказуемое линейное поведение, что соответствует проектным ожиданиям для асинхронной архитектуры.

При нагрузке 1000 RPS наблюдается качественный переход в зону нелинейного роста задержки: `proxy_total_ms` увеличивается до 10,52 мс, то есть в 3,5 раза по сравнению с 500 RPS. Как видно на рисунке 9, пиковые выбросы сквозной задержки достигают 50 мс: они обусловлены эпизодической задержкой установления TCP-соединения (`httpx_connection.connect_tcp_ms` достигает 26,65 мс на пиках) в момент выхода пула за предел кешированных соединений. Тем не менее фактический RPS на всех этапах совпадает с целевым, что подтверждает отсутствие отказов обслуживания: система справляется с нагрузкой за счёт увеличения задержки без потери пропускной способности.

Параметры. Для заглушки включён Rate Limiting с лимитом 250 запросов в 1 секунду. Нагрузочный генератор подаёт 500 RPS в течение 120 секунд.

Ожидаемый результат. В каждом секундном окне должно быть обслужено не более установленного лимита запросов; остальные должны получить код HTTP 429.

Наблюдаемые результаты. Значения счётчиков приведены в таблице 21. Распределение ответов по кодам состояния в ходе испытания приведено на рисунке 10.

Таблица 21 – Результаты проверки Rate Limiting под нагрузкой 500 RPS

Метрика	Значение
Установленный лимит (<code>rate_limit</code>), зап./сек	250
Всего запросов от нагрузочного генератора	60 042
Проксировано успешно (<code>proxy_total</code>)	30 039
Отклонено алгоритмом (<code>rejected_total</code>)	30 003
Доля отклонённых, %	50
Ожидаемое число принятых за 120 сек	30 000
Отклонение от теоретического значения, %	0,13



Рисунок 10 – Распределение ответов по кодам состояния
HTTP: Rate Limiting 250 зап./сек при нагрузке 500 RPS

Анализ результатов. Результаты испытания подтверждают корректность работы алгоритма Fixed Window Counter при нагрузке, вдвое превышающей установленный лимит. За 120 секунд нагрузочный генератор направил 60 042 запроса; из них проксировано 30 039, отклонено с кодом HTTP 429 – 30 003. Отклонение числа принятых запросов от теоретического значения ($30\,000 = 250 \text{ зап./сек} \times 120 \text{ сек}$) составило 0,13 %, что находится в пределах погрешности, обусловленной гранулярностью секундных окон и временным разрешением нагрузочного генератора.

Как видно на рисунке 10, в каждую секунду первые 250 запросов получают статус 200, последующие немедленно возвращают 429 без пересылки к заглушке, что исключает избыточную нагрузку на целевой сервис. Отсутствие выбросов за пределы лимита на протяжении всего 120-секундного окна подтверждает атомарность операции INCR в Redis: конкурентные запросы не создают состояние гонки при инкременте счётчика.

Необходимо учитывать известное ограничение алгоритма Fixed Window: на стыке двух соседних окон теоретически возможен кратковременный всплеск до $2 \times \text{limit}$ запросов в любой промежуток времени

длиной `window_size` [51]. В контексте данной системы это ограничение является приемлемым, поскольку целевой сервис (имитационная заглушка) не является высоконагруженным боевым сервисом, а применяемый лимит служит защитой от случайных перегрузок в тестовой инфраструктуре.

Сводная таблица результатов всех сценариев нагрузочного испытания приведена в таблице 22.

Таблица 22 – Сводные результаты нагрузочного испытания

Сценарий	Результат	Примечание
Базовое измерение при 10 RPS	Пройден	<code>proxy_total_ms</code> = 3,90 мс; накладные расходы прокси $\approx$ 1 мс
Нарастающая нагрузка до 1000 RPS	Пройден	Линейный рост до 500 RPS; нелинейный рост задержки при 1000 RPS без отказов
Rate Limiting при 500 RPS (лимит 250 зап./сек)	Пройден	Отклонение от теоретического лимита 0,13 %; атомарность INCR подтверждена

Общий анализ результатов. Совокупность проведённых нагрузочных испытаний позволяет сделать следующие выводы о производительностных характеристиках разработанного прокси-модуля.

Прокси-модуль обеспечивает приемлемую сквозную задержку в диапазоне нагрузок, характерных для тестовой инфраструктуры: при 125–500 RPS среднее значение `proxy_total_ms` не превышает 3,01 мс, что соответствует требованиям прозрачного проксирования. Переход в зону нелинейного роста задержки при 1000 RPS (до 10,52 мс) обусловлен насыщением пула HTTP-соединений `httpx`: при превышении числа кешированных соединений возникает необходимость установления новых TCP-сессий, накладные расходы которых (до 26 мс по метрике `httpx_connection.connect_tcp_ms`) существенно увеличивают сквозную задержку. Данная характеристика является ожидаемым поведением пула соединений при резком росте конкуренции и не свидетельствует об архитектурных ограничениях системы.

Алгоритм Fixed Window Counter демонстрирует высокую точность ограничения: отклонение от теоретического числа принятых запросов не превышает 0,2 % на 120-секундном горизонте. Атомарность операций Redis

INCR исключает состояние гонки при конкурентной обработке запросов. Совокупность приведённых результатов подтверждает соответствие реализованного программного комплекса заявленным функциональным требованиям в части производительности и корректности алгоритма управления потоком запросов.

3.4 Выводы

В третьей главе описан полный цикл внедрения, тестирования и эксплуатации разработанного программного комплекса управления инфраструктурой имитационных сервисов.

Раздел развёртывания охватывает требования к аппаратно-программной среде, процедуры установки зависимостей, генерации ключа шифрования, настройки конфигурационного файла и регистрации приложения в качестве системной службы `systemd` на платформе Astra Linux. Показано, что минимальная конфигурация (8 vCPU, 8 ГБ ОЗУ) достаточна для работы всех компонентов комплекса на одном управляющем узле.

Раздел функционального тестирования (5 сценариев) подтвердил корректность реализации двух ключевых механизмов отказоустойчивости: процедуры `restore_state` и планировщика `HealthChecker`. Во всех сценариях наблюдаемые результаты совпали с ожидаемыми: заглушки корректно восстанавливаются при перезапуске управляющего приложения (в том числе с автоматическим перезапуском завершившихся Java-процессов), данные реестра восстанавливаются после аварийного завершения Redis из AOF-журнала, а `HealthChecker` обнаруживает аварийное завершение заглушки и автоматически возвращает её в штатное состояние в пределах одного интервала проверки (≤ 30 сек).

Раздел нагрузочного тестирования (3 сценария) позволил количественно охарактеризовать производительность прокси-модуля и точность алгоритма Rate Limiting. Установлено, что при нагрузках до 500 RPS сквозная задержка не превышает 3,01 мс и растёт линейно; при 1000 RPS наблюдается

нелинейный рост до 10,52 мс вследствие насыщения пула TCP-соединений, однако без потери пропускной способности. Алгоритм Fixed Window Counter на базе атомарных операций Redis обеспечивает точность ограничения с отклонением не более 0,2 % от теоретического значения, что подтверждает корректность его реализации в условиях конкурентной нагрузки.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы спроектирован и реализован программный комплекс для централизованного управления и мониторинга распределённой инфраструктуры имитационных сервисов. Все поставленные задачи решены в полном объёме.

В ходе анализа предметной области установлено, что существующие зарубежные решения (WireMock, MockServer, Mountebank) не обеспечивают совместимость с отечественными сертифицированными операционными системами семейства Astra Linux и не поддерживают централизованное управление распределённой инфраструктурой Java-процессов без установки агентного программного обеспечения на целевые серверы. Сравнительный анализ инструментальных средств позволил обосновать технологический стек: Python / FastAPI / Redis, – оптимально сочетающий асинхронную модель обработки запросов с минимальной задержкой доступа к данным конфигурации.

При проектировании архитектуры системы выбрана монолитная организация с явно выраженной внутренней модульностью, формализованная в нотации C4 Model. Разработана агентно-независимая топология взаимодействия с удалёнными хостами через протоколы SSH/SFTP, исключающая необходимость предварительной установки дополнительного программного обеспечения на целевые серверы.

Реализованы алгоритм динамического проксирования HTTP-трафика с сохранением метода, заголовков и тела запроса, а также алгоритм ограничения интенсивности запросов Fixed Window Counter на основе атомарных операций Redis. Разработан механизм восстановления состояния системы при перезапуске: процедура `restore_state` сверяет метаданные реестра с фактическим состоянием операционной системы и автоматически перезапускает завершившиеся Java-процессы. Реализован модуль экспорта метрик в формате

Prometheus с двухпроходной конвейерной схемой сборки данных за фиксированное число обращений к Redis.

Развёртывание и тестирование системы выполнены в среде ОС Astra Linux. По итогам пяти функциональных сценариев подтверждена корректность механизмов персистентности данных (AOF-восстановление Redis) и отказоустойчивости (обнаружение аварийного завершения загрузок планировщиком HealthChecker в пределах 30 секунд). Нагрузочное тестирование показало, что прокси-модуль обеспечивает сквозную задержку не более 3,01 мс при нагрузках до 500 RPS, а алгоритм Rate Limiting удерживает точность ограничения с отклонением 0,13 % от теоретического значения.

Практическая значимость работы определяется созданием программного инструмента, позволяющего отказаться от зарубежных решений в задачах управления инфраструктурой имитационных сервисов и обеспечивающего интеграцию со стандартными средствами мониторинга. Направлениями дальнейшего развития системы могут стать поддержка сценариев динамической маршрутизации запросов на основе содержимого тела, реализация ролевой модели разграничения доступа к управляющему интерфейсу, а также расширение поддерживаемых типов артефактов за пределы JVM-платформы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р 56920–2024. Системная и программная инженерия. Тестирование программного обеспечения. Общие положения: Техн. отчет ГОСТ Р 56920-2024. – М.: Российский институт стандартизации, 2024.
2. Куликов, Святослав. Тестирование программного обеспечения. Базовый курс / Святослав Куликов. – 3-е изд. изд. – ЕРАМ Systems, 2025. – 301 с.
3. Буренин, П. В. Безопасность операционной системы специального назначения Astra Linux Special Edition / П. В. Буренин, П. Н. Девянин, В. А. Лыфарь, А. А. Малофеев. – 3-е изд., перераб. и доп. изд. – М.: Горячая линия – Телеком, 2019. – 368 с.
4. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления: Техн. отчет ГОСТ 7.32-2017. – М.: Стандартинформ, 2018.
5. Ньюмен, Сэм. Создание микросервисов / Сэм Ньюмен. – СПб.: Питер, 2016. – 304 с.
6. WildFly – JBoss Application Server Documentation. – 2025. – URL: <https://www.wildfly.org/documentation/> (дата обращения: 15.02.2026). – Текст: электронный.
7. Eclipse GlassFish Documentation. – 2025. – URL: <https://glassfish.org/documentation> (дата обращения: 15.02.2026). – Текст: электронный.
8. Hattingh, Caleb. Using Asyncio in Python: Understanding Python’s Asynchronous Programming Features / Caleb Hattingh. – Sebastopol: O’Reilly Media, 2020. – 166 с.
9. Рамальо, Лусиану. Python. К вершинам мастерства: Лаконичное и эффективное программирование / Лусиану Рамальо. – 2-е изд. изд. – М.: ДМК Пресс, 2022. – 898 с.

10. Beazley, David. Understanding the Python GIL. – 2010. – PyCon 2010, Atlanta, Georgia. – URL: <http://www.dabeaz.com/python/GIL.pdf> (дата обращения: 15.02.2026). – Текст: электронный.
11. Beazley, David. Python Cookbook / David Beazley, Brian K. Jones. – 3rd ed. edition. – Sebastopol: O'Reilly Media, 2013. – 706 с.
12. Гринберг, Мигель. Разработка веб-приложений с использованием Flask на языке Python / Мигель Гринберг. – М.: ДМК Пресс, 2014. – 272 с.
13. Kerrisk, Michael. The Linux Programming Interface: A Linux and UNIX System Programming Handbook / Michael Kerrisk. – San Francisco: No Starch Press, 2010. – 1552 с.
14. FastAPI Documentation. – 2025. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 15.02.2026). – Текст: электронный.
15. Любанович, Билл. FastAPI: веб-разработка на Python / Билл Любанович. – Астана: Спринт Бук, 2024. – 288 с.
16. ASGI (Asynchronous Server Gateway Interface) Specification. – 2025. – URL: <https://asgi.readthedocs.io/en/latest/> (дата обращения: 15.02.2026). – Текст: электронный.
17. Pydantic Documentation – Data validation using Python type hints. – 2025. – URL: <https://docs.pydantic.dev/> (дата обращения: 15.02.2026). – Текст: электронный.
18. Uvicorn – An ASGI web server, for Python. – 2025. – URL: <https://www.uvicorn.org/> (дата обращения: 15.02.2026). – Текст: электронный.
19. Моргунов, Евгений Павлович. PostgreSQL. Основы языка SQL / Евгений Павлович Моргунов. – СПб.: БХВ-Петербург, 2018. – 336 с.
20. Садаладж, Прамод Дж. NoSQL: новая методология разработки нереляционных баз данных / Прамод Дж. Садаладж, Мартин Фаулер. – М.: Вильямс, 2013. – 192 с.
21. Redis Documentation. – 2025. – URL: <https://redis.io/docs/> (дата обращения: 15.02.2026). – Текст: электронный.

22. Macedo, Tiago. Redis Cookbook / Tiago Macedo, Fred Oliveira. – Sebastopol: O'Reilly Media, 2011. – 74 с.
23. Redis Persistence: Append Only File. – 2025. – URL: <https://redis.io/docs/management/persistence/> (дата обращения: 15.02.2026). – Текст: электронный.
24. Клеппман, Мартин. Высоконагруженные приложения. Программирование, масштабирование, поддержка / Мартин Клеппман. – СПб.: Питер, 2018. – 640 с.
25. Astra Linux Special Edition – официальная документация. – 2025. – URL: <https://wiki.astralinux.ru/> (дата обращения: 15.02.2026). – Текст: электронный.
26. WireMock Documentation. – 2025. – URL: <https://wiremock.org/docs/> (дата обращения: 09.01.2026). – Текст: электронный.
27. MockServer Documentation. – 2025. – URL: <https://www.mock-server.com/> (дата обращения: 09.01.2026). – Текст: электронный.
28. Mountebank Documentation. – 2025. – URL: <http://www.mbtest.org/docs/> (дата обращения: 09.01.2026). – Текст: электронный.
29. Apache Tomcat 9 Documentation. – 2025. – URL: <https://tomcat.apache.org/tomcat-9.0-doc/index.html> (дата обращения: 09.01.2026). – Текст: электронный.
30. Tomcat Clustering/Session Replication How-To. – 2025. – URL: <https://tomcat.apache.org/tomcat-9.0-doc/cluster-howto.html> (дата обращения: 09.01.2026). – Текст: электронный.
31. Apache Tomcat Configuration Reference: The Rate Limit Filter. – 2025. – URL: https://tomcat.apache.org/tomcat-9.0-doc/config/filter.html#Rate_Limit_Filter (дата обращения: 09.01.2026). – Текст: электронный.
32. Вигерс, Карл. Разработка требований к программному обеспечению / Карл Вигерс, Джой Битти. – 3-е изд., доп. изд. – М.; СПб.: Русская Редакция; БХВ-Петербург, 2014. – 736 с.

33. ГОСТ 34.602–2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы: Техн. отчет ГОСТ 34.602-2020. – М.: Российский институт стандартизации, 2021.

34. ГОСТ Р ИСО/МЭК 25010–2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения: Техн. отчет ГОСТ Р ИСО/МЭК 25010-2015. – М.: Стандартиформ, 2015.

35. Матвеев, М. Д. Astra Linux. Установка, настройка, администрирование / М. Д. Матвеев. – СПб.: Наука и Техника, 2023.

36. OpenJDK 17 Documentation. – 2025. – URL: <https://openjdk.org/projects/jdk/17/> (дата обращения: 15.02.2026). – Текст: электронный.

37. Фаулер, Мартин. Архитектура корпоративных программных приложений / Мартин Фаулер. – М.: Издательский дом «Вильямс», 2006. – 544 с.

38. ГОСТ Р 59793–2021. Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания: Техн. отчет ГОСТ Р 59793-2021. – М.: Российский институт стандартизации, 2021.

39. The C4 model for visualising software architecture. – 2025. – URL: <https://c4model.com/> (дата обращения: 15.02.2026). – Текст: электронный.

40. Brown, Simon. Software Architecture for Developers. Vol. 2: Visualise, document and explore your software architecture / Simon Brown. – Leanpub, 2019. – 193 с.

41. Jinja2 Documentation – The Python Template Engine. – 2025. – URL: <https://jinja.palletsprojects.com/> (дата обращения: 15.02.2026). – Текст: электронный.

42. Paramiko Documentation – SSH2 protocol library for Python. – 2025. – URL: <https://www.paramiko.org/> (дата обращения: 15.02.2026). – Текст: электронный.

43. HTTPX – A next-generation HTTP client for Python. – 2025. – URL: <https://www.python-httpx.org/> (дата обращения: 15.02.2026). – Текст: электронный.
44. Cryptography Library – Fernet (symmetric encryption). – 2025. – URL: <https://cryptography.io/en/latest/fernet/> (дата обращения: 15.02.2026). – Текст: электронный.
45. Prometheus Documentation. – 2025. – URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 15.02.2026). – Текст: электронный.
46. Prometheus Exposition Formats. – 2025. – URL: https://prometheus.io/docs/instrumenting/exposition_formats/ (дата обращения: 15.02.2026). – Текст: электронный.
47. Turnbull, James. Monitoring with Prometheus / James Turnbull. – James Turnbull, 2018. – 392 с.
48. Redis Commands: INCR – Increment the integer value of a key. – 2025. – URL: <https://redis.io/commands/incr/> (дата обращения: 15.02.2026). – Текст: электронный.
49. Web Content Accessibility Guidelines (WCAG) 2.1. – 2018. – URL: <https://www.w3.org/TR/WCAG21/> (дата обращения: 15.02.2026). – Текст: электронный.
50. Fielding, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures: Диссертация / University of California, Irvine. – 2000. – 162 с.
51. Таненбаум, Эндрю. Компьютерные сети / Эндрю Таненбаум, Дэвид Уэзеролл. – 5-е изд. изд. – СПб.: Питер, 2012. – 960 с.
52. Французов, А. М. Обзор и сравнительный анализ современных средств нагрузочного тестирования веб-приложений / А. М. Французов // Новые тенденции в науке, обществе и технике: сборник статей Международной научно-практической конференции. – Ульяновск: ИП Кеньшенская Виктория Валерьевна (издательство «Зебра»), 2025. – С. 284–292.

ПРИЛОЖЕНИЕ А

Листинг А.1 – Запуск Java-процесса на локальном хосте

```
1 async def start_mock(self, filename: str) -> None:
2     if self._is_local(hostname):
3         port = await self._pick_free_port_local(pmin, pmax)
4         log_file_path = _log_path_for_filename(filename)
5         # Открытие лог-файла вынесено в поток (I/O-bound операция)
6         lf = await asyncio.to_thread(open, log_file_path, "ab")
7         try:
8             argv = [java_path]
9             argv.extend(shlex.split(jvm_args) if jvm_args.strip()
10                ↪ else [])
11             argv.extend(["-jar", artifact_path,
12                ↪ f"--server.port={port}"])
13             # Неблокирующий запуск дочернего процесса
14             proc = await asyncio.create_subprocess_exec(
15                 *argv,
16                 stdin=asyncio.subprocess.DEVNULL,
17                 stdout=lf,
18                 stderr=asyncio.subprocess.STDOUT,
19                 start_new_session=True)
20         finally:
21             lf.close()
22         pid = proc.pid
```


Листинг А.2 – Формирование SSH-команды запуска Java-процесса на удалённом хосте

```
1 # Экранирование JVM-аргументов для безопасной передачи через SSH
2 jvm_part = " ".join(
3     shlex.quote(t) for t in shlex.split(jvm_args.strip())
4 ) if jvm_args.strip() else ""

5 mid = f"{jvm_part} " if jvm_part else ""

6 # Итоговая команда: nohup, перенаправление вывода, вывод PID
7 cmd = (
8     f"nohup {shlex.quote(java_path)} {mid}"
9     f"-jar {shlex.quote(artifact_path)} --server.port={int(port)} "
10    f">> {shlex.quote(log_remote)} 2>&1 & echo $!"
11 )
12 proc = await self._ssh.run_command(
13     hostname, user, pwd, cmd, port=ssh_port, timeout=60.0
14 )
15 # Последняя строка вывода содержит PID запущенного процесса
16 pid = int(proc.stdout.strip().splitlines()[-1])
```

Листинг А.3 – Методы проверки живости процесса

```
1 # Локальная проверка: системный вызов в пуле потоков
2 async def _process_alive_local(self, pid: int) -> bool:
3     def check() -> bool:
4         try:
5             os.kill(pid, 0) # сигнал 0: только проверка
6                 ↪ существования
7         except OSError:
8             return False # процесс не найден или нет прав
9         return True
10
11     return await asyncio.to_thread(check)
12
13 # Удалённая проверка: команда kill -0 через SSH
14 async def _process_alive_remote(
15     self, hostname: str, username: str, password: str,
16     ssh_port: int, pid: int,
17 ) -> bool:
18     proc = await self._ssh.run_command(
19         hostname, username, password,
20         f"kill -0 {int(pid)} 2>/dev/null && echo OK || echo NO",
21         port=ssh_port, timeout=15.0,
22     )
23     return (proc.stdout or "").strip().endswith("OK")
```

Листинг А.4 – Реестр HTTP-клиентов ProxyClientRegistry

```
1 class ProxyClientRegistry:
2     """Один httpx.AsyncClient на заглушку с пулом соединений."""
3     def __init__(self) -> None:
4         self._clients: dict[str, httpx.AsyncClient] = {}
5         self._lock = asyncio.Lock()
6     async def get_or_create(
7         self,
8         mock_name: str,
9         base_url: str,
10        timeout: float | httpx.Timeout,
11    ) -> httpx.AsyncClient:
12        async with self._lock:
13            existing = self._clients.get(mock_name)
14            if existing is not None:
15                return existing
16            limits = httpx.Limits(
17                max_connections=1000,
18                max_keepalive_connections=200,
19                keepalive_expiry=60.0)
20            t = timeout if isinstance(timeout, httpx.Timeout) else
21                ↪ httpx.Timeout(timeout)
22            client = httpx.AsyncClient(
23                base_url=base_url.rstrip("/"),
24                timeout=t,
25                limits=limits,
26                follow_redirects=False)
27            self._clients[mock_name] = client
28            return client
29    async def remove(self, mock_name: str) -> None:
30        async with self._lock:
31            client = self._clients.pop(mock_name, None)
32            if client is not None:
33                await client.aclose()    # корректное закрытие всех
34                ↪ соединений пула
```

Листинг A.5 – Реализация алгоритма Fixed Window Counter

```
1 class RateLimiter:
2     """Ограничение частоты запросов на заглушку в фиксированных
   ↪ временных окнах."""
3
4     def __init__(self, redis: Redis) -> None:
5         self._redis = redis
6
7     async def check(self, filename: str, limit: int,
8                     window_size: int = 1) -> bool:
9         """Инкрементирует счётчик текущего окна.
10        Возвращает True, если запрос в пределах лимита."""
11        window = int(time.time()) // window_size
12        key = f"rate:{filename}:{window}"
13
14        # Атомарный инкремент: Redis гарантирует неделимость
15        ↪ операции
16        count = await self._redis.incr(key)
17
18        # TTL устанавливается только при создании нового счётчика
19        ↪ (count == 1)
20        if count == 1:
21            await self._redis.expire(key, window_size * 2)
22
23        return count <= limit # True -> разрешить, False -> HTTP
24        ↪ 429
```

Листинг А.6 – Маршрут панели управления: конвейерная выборка заглушек из Redis

```
1 @router.get("/", include_in_schema=False)
2 async def dashboard_page(request: Request, redis: RedisClientDep,
    ↪ ...):
3     names = sorted(await redis.smembers("mocks:registry"))
4     if names:
5         # Pipeline: одна сетевая транзакция вместо N отдельных
        ↪ HGETALL
6         async with redis.pipeline(transaction=False) as pipe:
7             for name in names:
8                 pipe.hgetall(f"mocks:{name}")
9                 rows = await pipe.execute()
10    return templates.TemplateResponse(request=request,
    ↪ name="dashboard.html",
11        context={"mocks": mocks, "mocks_total": len(mocks),
12                "mocks_running": running, "redis_connected": ...,
13                "active_page": "dashboard"})
```

Листинг А.7 – Двухпроходная конвейерная сборка метрик из Redis

```
1 async def collect_metrics(redis: Redis) -> str:
2     # Проход 1: реестры
3     pipe = redis.pipeline(transaction=False)
4     pipe.smembers("mocks:registry")
5     pipe.smembers("hosts:registry")
6     reg_results = await pipe.execute()
7     mock_files = sorted(reg_results[0] or [])
8     host_names = sorted(reg_results[1] or [])

9     # Проход 2: данные заглушек, хостов и все метрические ключи
10    pipe2 = redis.pipeline(transaction=False)
11    for filename in mock_files:
12        pipe2.hgetall(f"mocks:{filename}")          # статус заглушки
13    for hostname in host_names:
14        pipe2.hget(f"hosts:{hostname}", "status")    # статус хоста
15    for filename in mock_files:
16        pipe2.get(f"metrics:proxy_total:{filename}")
17        pipe2.get(f"metrics:rejected_total:{filename}")
18        pipe2.hgetall(f"metrics:proxy_stage_sum_ms:{filename}")
19        pipe2.hgetall(f"metrics:proxy_stage_count:{filename}")
20        pipe2.hgetall(f"metrics:proxy_stage_last_ms:{filename}")
21        pipe2.hgetall(f"metrics:proxy_outcome_total:{filename}")
22        pipe2.hgetall(f"metrics:proxy_upstream_status_total:{filename}")
23    # Агрегаты __all__
24    pipe2.hgetall("metrics:proxy_stage_sum_ms:__all__")
25    pipe2.hgetall("metrics:proxy_stage_count:__all__")
26    pipe2.hgetall("metrics:proxy_stage_last_ms:__all__")
27    pipe2.hgetall("metrics:proxy_outcome_total:__all__")
28    pipe2.hgetall("metrics:proxy_upstream_status_total:__all__")
29    batch = await pipe2.execute()
30    # ... формирование строк текстового ответа ...
```

Листинг А.8 – Фрагмент ответа маршрута /metrics

```
1 mockcontrol_proxy_stage_latency_ms_count{mock="__all__",stage="res  
  ↳ ponse_build_ms"} 158287  
2 mockcontrol_proxy_stage_latency_ms_avg{mock="__all__",stage="respo  
  ↳  nse_build_ms"} 0.008  
3 mockcontrol_proxy_stage_latency_ms_last{mock="__all__",stage="resp  
  ↳  onse_build_ms"} 0.008  
4 mockcontrol_proxy_outcome_total{mock="uuid-stub-jdk8-2.1.war",outc  
  ↳  ome="ok"} 1  
5 mockcontrol_proxy_outcome_total{mock="uuid-stub-jdk8-fixed-local.w  
  ↳  ar",outcome="ok"} 158270  
6 mockcontrol_proxy_outcome_total{mock="__all__",outcome="not_found"}  
  ↳  3
```

Листинг А.9 – Файл юнита systemd mockcontrol.service

```
1 [Unit]
2 Description=MockControl – управляющий сервис инфраструктуры
   ↳ имитационных сервисов
3 After=network.target redis.service
4 Wants=redis.service

5 [Service]
6 KillMode=process
7 Type=simple
8 User=klim
9 WorkingDirectory=/app/educ-thesis/

10 # Путь до интерпретатора Python из виртуального окружения.
11 # Если venv не используется – замените на просто: /usr/bin/python3
12 ExecStart=/app/educ-thesis/.venv/bin/python main.py

13 # Перезапуск при аварийном завершении (ненулевой код выхода)
14 Restart=on-failure
15 RestartSec=5

16 # Переменные окружения (альтернатива .env – можно указать здесь)
17 EnvironmentFile=/app/educ-thesis/.env

18 # Стандартный вывод и ошибки – в systemd journal
19 StandardOutput=journal
20 StandardError=journal
21 SyslogIdentifier=mockcontrol

22 # Немного времени на graceful shutdown перед SIGKILL
23 TimeoutStopSec=15

24 [Install]
25 WantedBy=multi-user.target
```


Листинг А.10 – Сценарий 1. Журнал службы mockcontrol: завершение процедуры restore_state – все заглушки успешно подтверждены

```
1 май 09 22:13:24 thesis-master systemd[1]: Started MockControl.
2 май 09 22:13:25 thesis-master mockcontrol[21458]: INFO:      Started
   ↳ server process [21458]
3 май 09 22:13:25 thesis-master mockcontrol[21458]: INFO:      Waiting
   ↳ for application startup.
4 май 09 22:13:25 thesis-master mockcontrol[21458]: 2026-05-09
   ↳ 22:13:25,008 INFO core.redis_client: Redis: pool connected
   ↳ (redis://192.168.1.76:6379/0)
5 май 09 22:13:25 thesis-master mockcontrol[21458]: 2026-05-09
   ↳ 22:13:25,010 INFO dependencies: Application dependencies
   ↳ initialized.
6 май 09 22:13:25 thesis-master mockcontrol[21458]: 2026-05-09
   ↳ 22:13:25,141 INFO services.lifecycle: restore_state: process
   ↳ alive, proxy client restored: uuid-stub-jdk8-2.war
7 май 09 22:13:25 thesis-master mockcontrol[21458]: 2026-05-09
   ↳ 22:13:25,155 INFO services.lifecycle: restore_state: process
   ↳ alive, proxy client restored: uuid-stub-jdk8-1.war
8 май 09 22:13:25 thesis-master mockcontrol[21458]: INFO:
   ↳ Application startup complete.
9 май 09 22:13:25 thesis-master mockcontrol[21458]: INFO:      Uvicorn
   ↳ running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Листинг А.11 – Сценарий 2. Журнал службы: обнаружение завершённых процессов и автоматический перезапуск заглушек в ходе restore_state

```
1 май 09 22:16:15 thesis-master systemd[1]: Started MockControl.
2 май 09 22:16:16 thesis-master mockcontrol[21494]: INFO:      Started
  ↳ server process [21494]
3 май 09 22:16:16 thesis-master mockcontrol[21494]: INFO:      Waiting
  ↳ for application startup.
4 май 09 22:16:16 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:16,611 INFO core.redis_client: Redis: pool connected
  ↳ (redis://192.168.1.76:6379/0)
5 май 09 22:16:16 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:16,612 INFO dependencies: Application dependencies
  ↳ initialized.
6 май 09 22:16:18 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:18,658 INFO services.lifecycle: Mock started:
  ↳ uuid-stub-jdk8-1.war pid=21501 port=8100
7 май 09 22:16:18 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:18,658 INFO services.lifecycle: restore_state: process
  ↳ restarted: uuid-stub-jdk8-1.war
8 май 09 22:16:20 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:20,806 INFO services.lifecycle: Mock started:
  ↳ uuid-stub-jdk8-2.war pid=27113 port=8100
9 май 09 22:16:20 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:20,806 INFO services.lifecycle: restore_state: process
  ↳ restarted: uuid-stub-jdk8-2.war
10 май 09 22:16:20 thesis-master mockcontrol[21494]: INFO:
  ↳ Application startup complete.
11 май 09 22:16:20 thesis-master mockcontrol[21494]: 2026-05-09
  ↳ 22:16:20,809 INFO main: Singleton health_checker: lock acquired
  ↳ by pid=21494
12 май 09 22:16:20 thesis-master mockcontrol[21494]: INFO:      Uvicorn
  ↳ running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Листинг А.12 – Сценарий 3. Журнал файла /var/log/redis/redis-server.log, подтверждающий старт из AOF после аварийного завершения

```
1 21558:M 09 May 2026 22:22:15.668 # Server initialized
2 21558:M 09 May 2026 22:22:15.669 * Reading RDB preamble from AOF
  ↳ file...
3 21558:M 09 May 2026 22:22:15.669 * Reading the remaining AOF tail...
4 21558:M 09 May 2026 22:22:16.474 * DB loaded from append only file:
  ↳ 0.806 seconds
5 21558:M 09 May 2026 22:22:16.474 * Ready to accept connections
```

Листинг А.13 – Сценарий 4. Журнал службы mockcontrol: заглушка переведена в статус ERROR после обнаружения аварийного завершения процесса

```
1 май 09 22:30:51 thesis-master mockcontrol[21494]: 2026-05-09
↳ 22:30:51,982 WARNING services.health_checker: Mock
↳ 'uuid-stub-jdk8-1.war': HTTP GET to root failed
↳ ('192.168.1.76':8100), attempt 1/3
2 май 09 22:31:22 thesis-master mockcontrol[21494]: 2026-05-09
↳ 22:31:22,014 WARNING services.health_checker: Mock
↳ 'uuid-stub-jdk8-1.war': process not found (pid=21501 on
↳ '192.168.1.76')
3 май 09 22:31:52 thesis-master mockcontrol[21494]: 2026-05-09
↳ 22:31:52,047 WARNING services.health_checker: Mock
↳ 'uuid-stub-jdk8-1.war': process not found (pid=21501 on
↳ '192.168.1.76')
```

Листинг А.14 – Сценарий 5. Журнал службы mockcontrol: заглушка вернулась в состояние RUNNING

```
1 май 09 22:34:44 thesis-master mockcontrol[21494]: 2026-05-09
↳ 22:34:44,828 INFO services.lifecycle: Mock started:
↳ uuid-stub-jdk8-1.war pid=21616 port=8100
```

ПРИЛОЖЕНИЕ Б

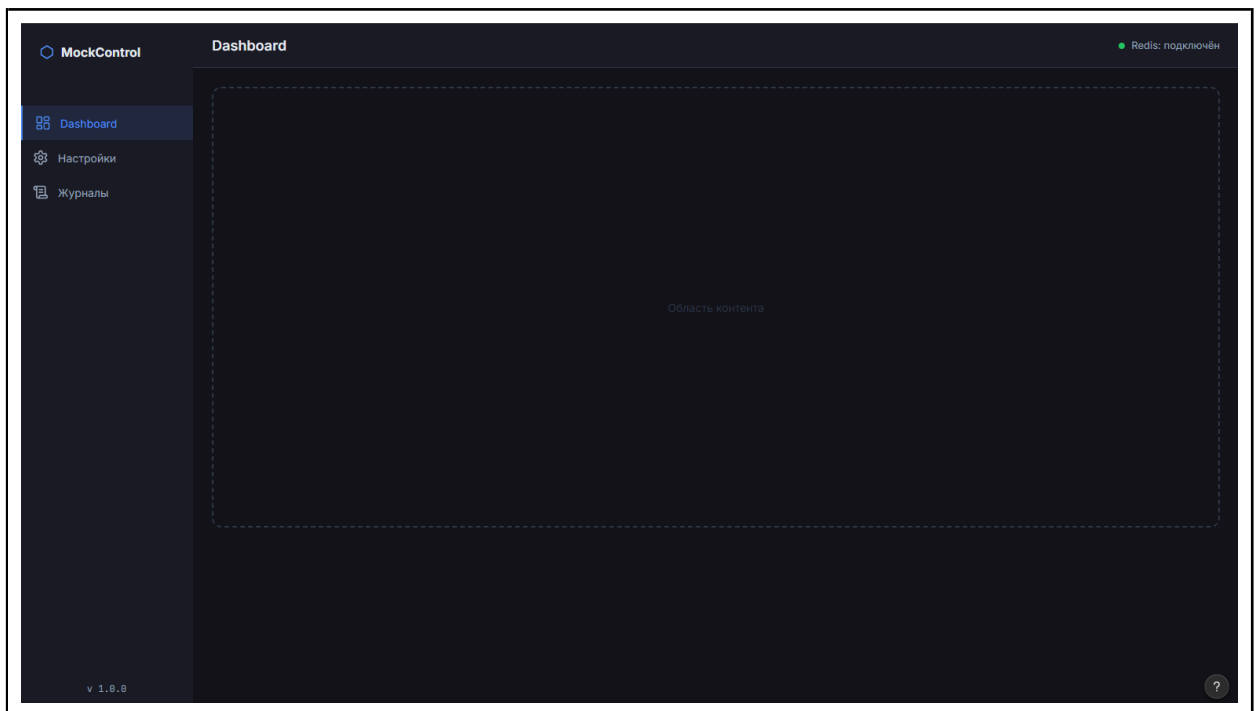


Рисунок Б.1 – Макет общего каркаса приложения с боковой навигацией

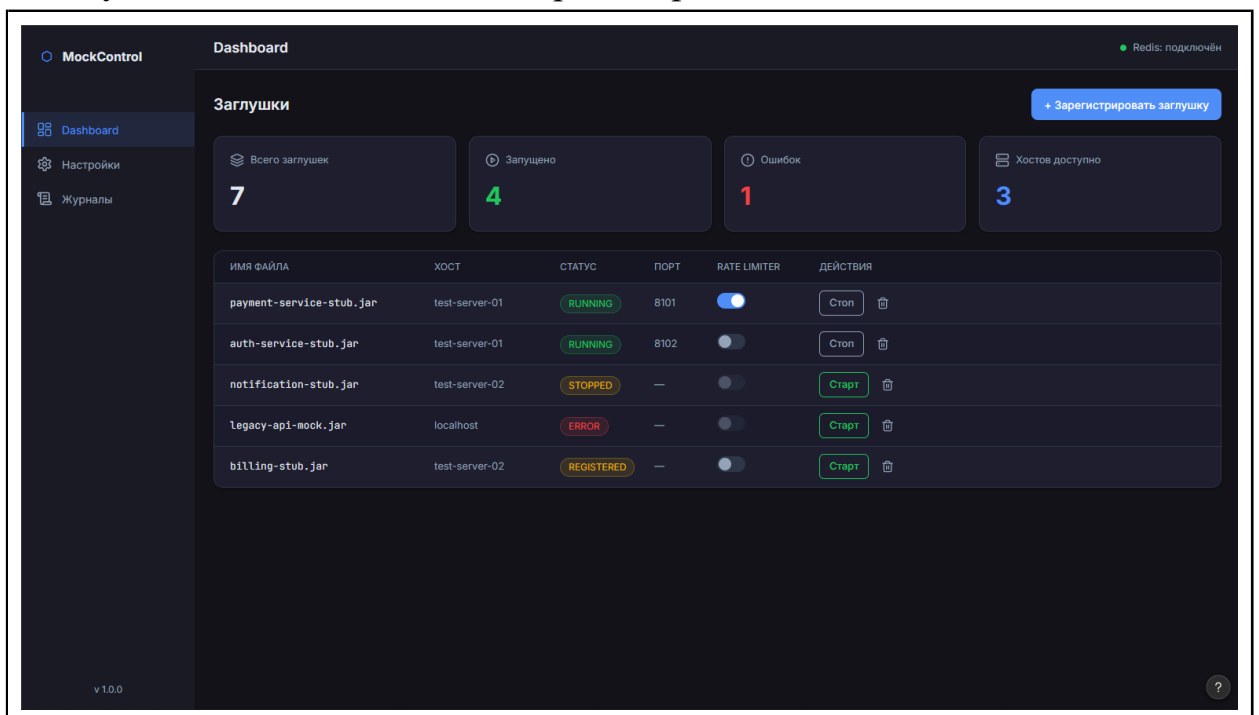


Рисунок Б.2 – Макет главной панели управления

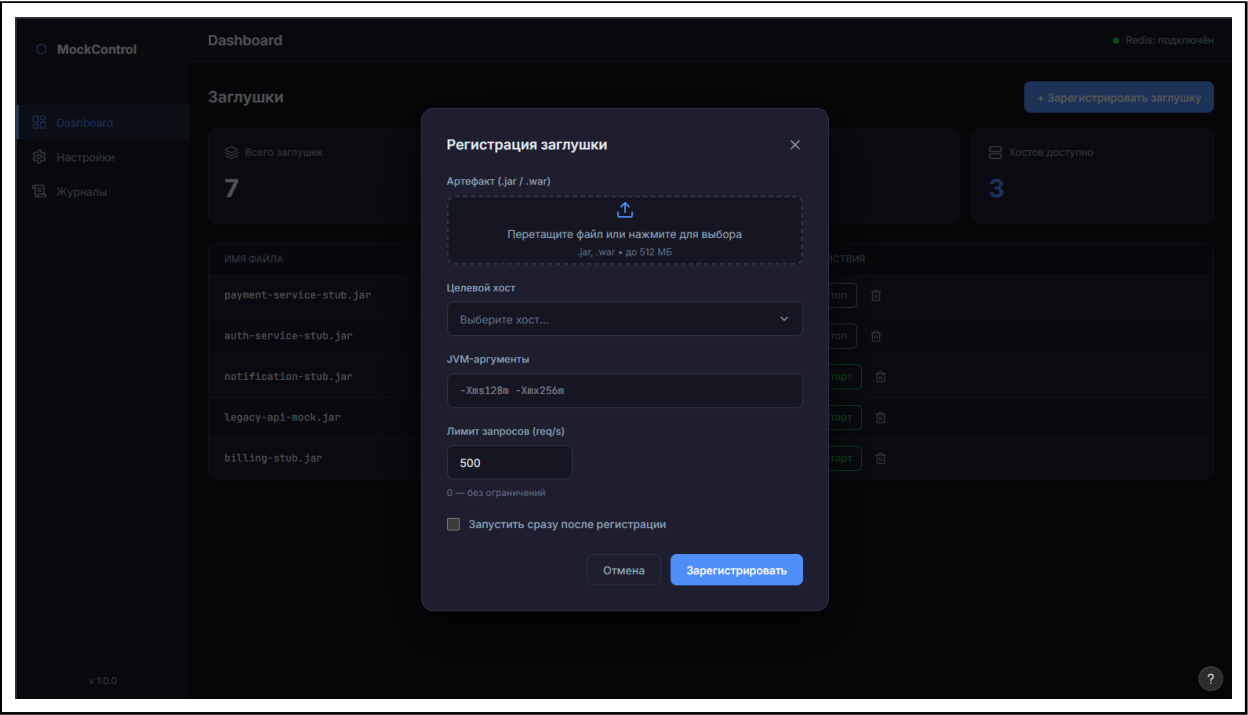


Рисунок Б.3 – Макет модального окна регистрации новой заглушки

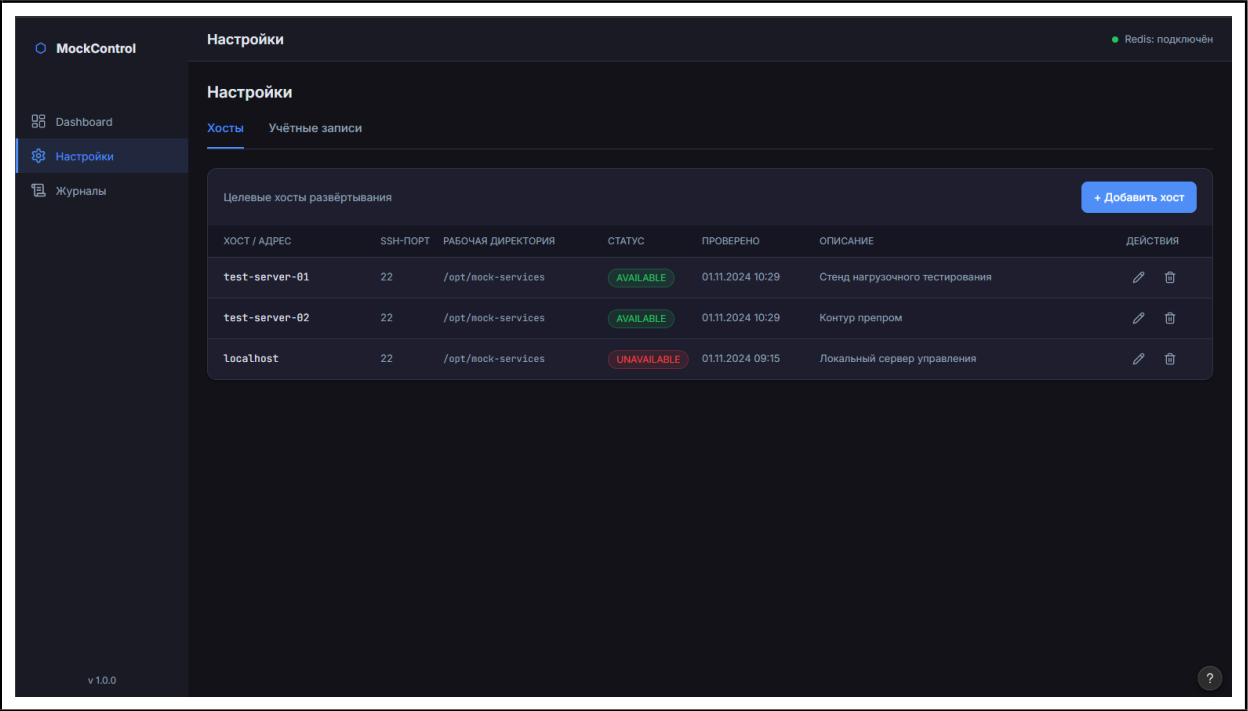


Рисунок Б.4 – Макет страницы настроек, вкладка «Хосты»

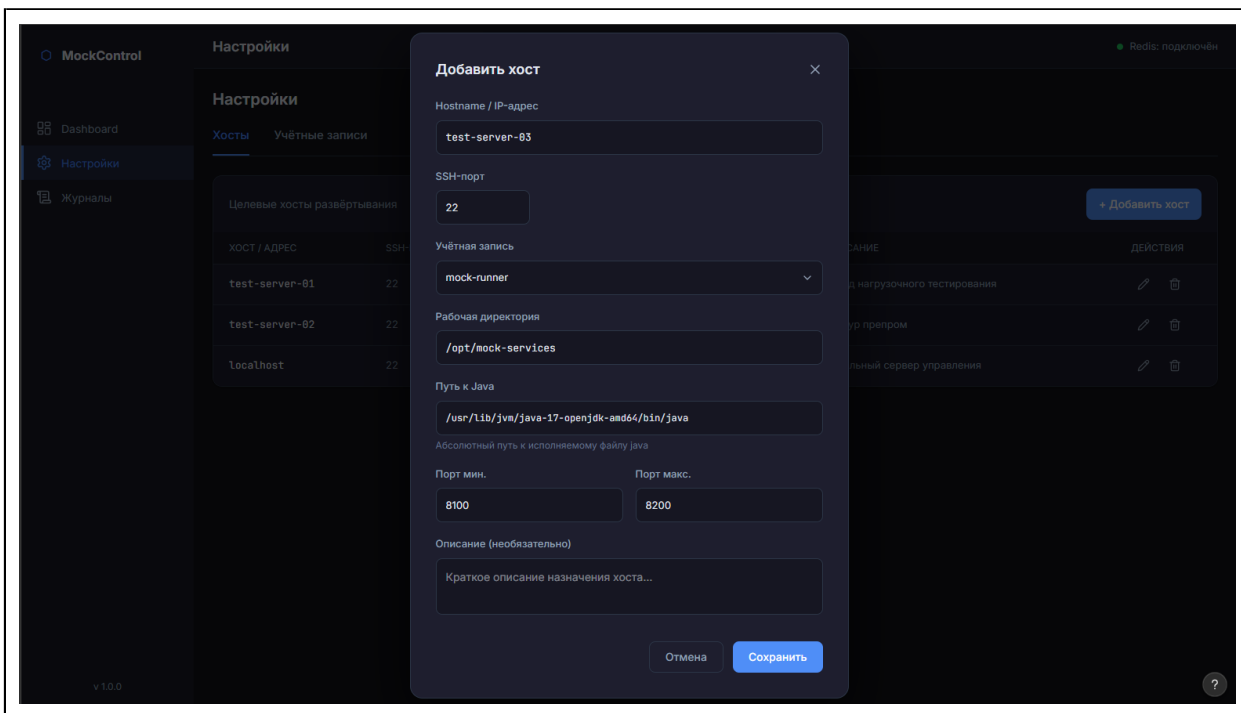


Рисунок Б.5 – Макет модального окна добавления нового хоста

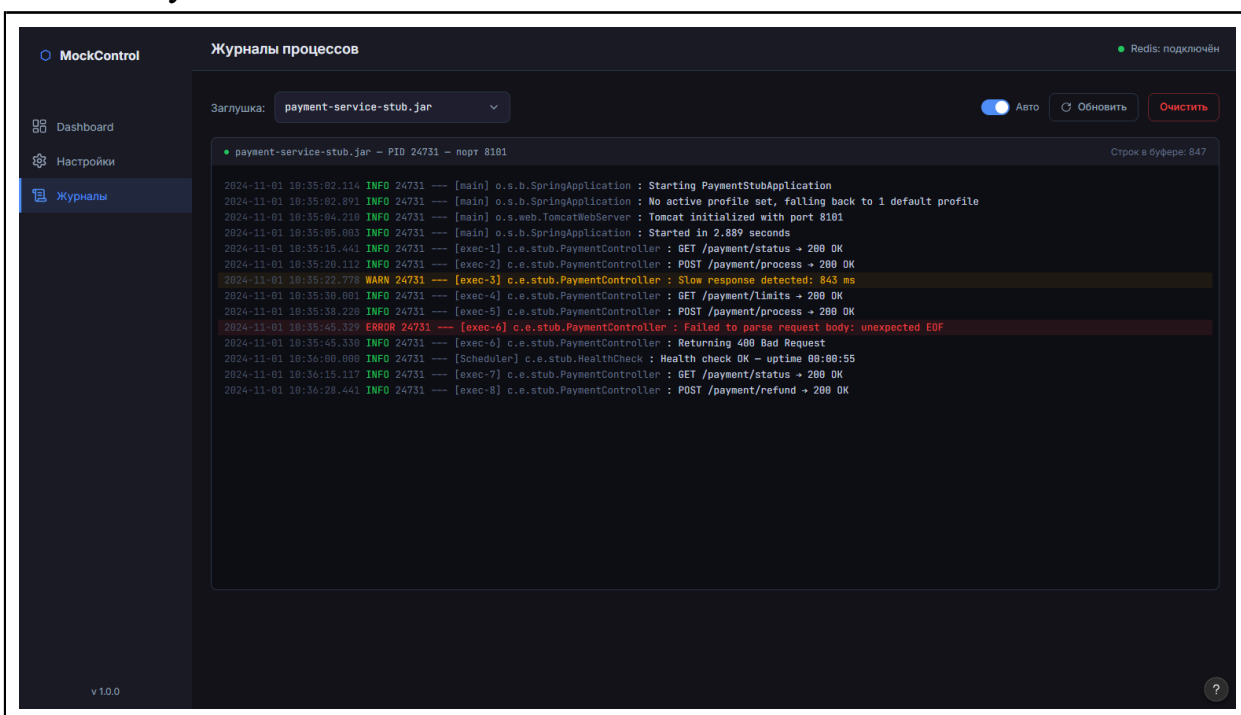


Рисунок Б.6 – Макет страницы просмотра журналов процессов

ПРИЛОЖЕНИЕ В

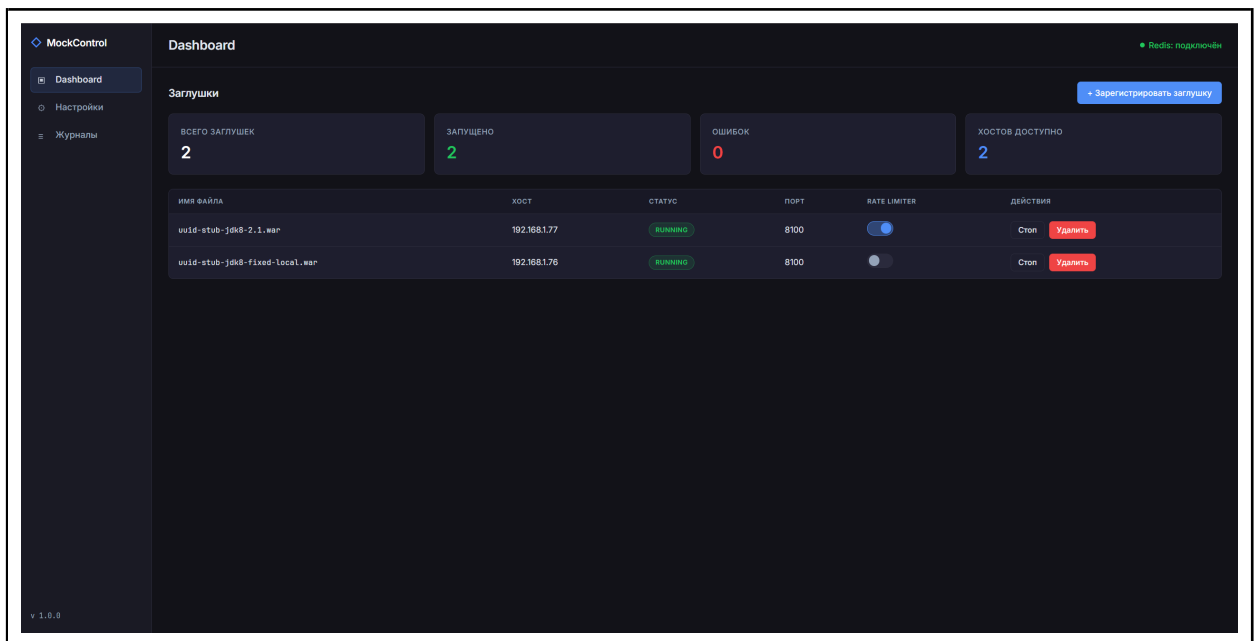


Рисунок В.1 – Реализованная главная панель управления

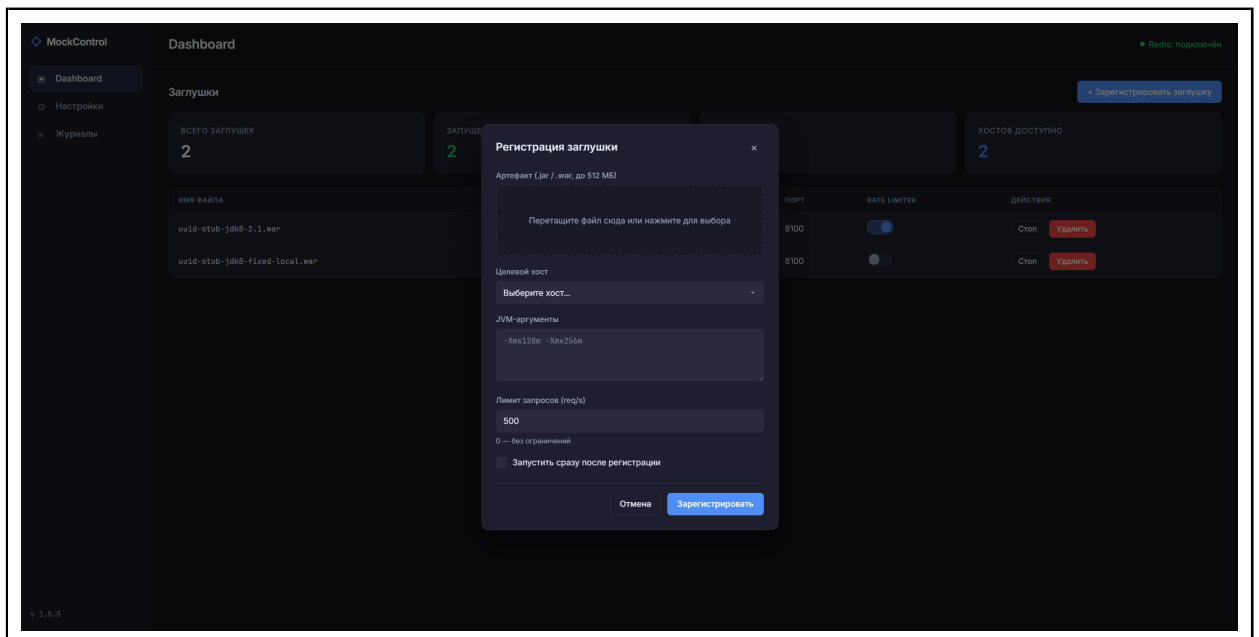


Рисунок В.2 – Реализованное диалоговое окно регистрации заглушки

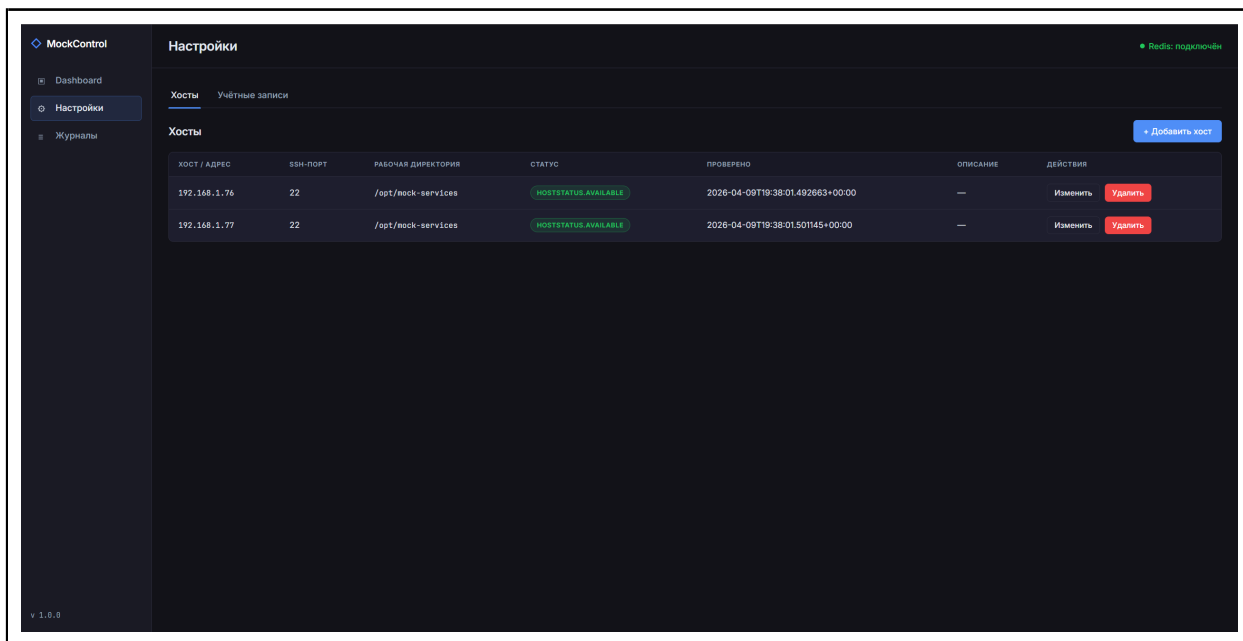


Рисунок В.3 – Реализованная страница настроек, вкладка «Хосты»

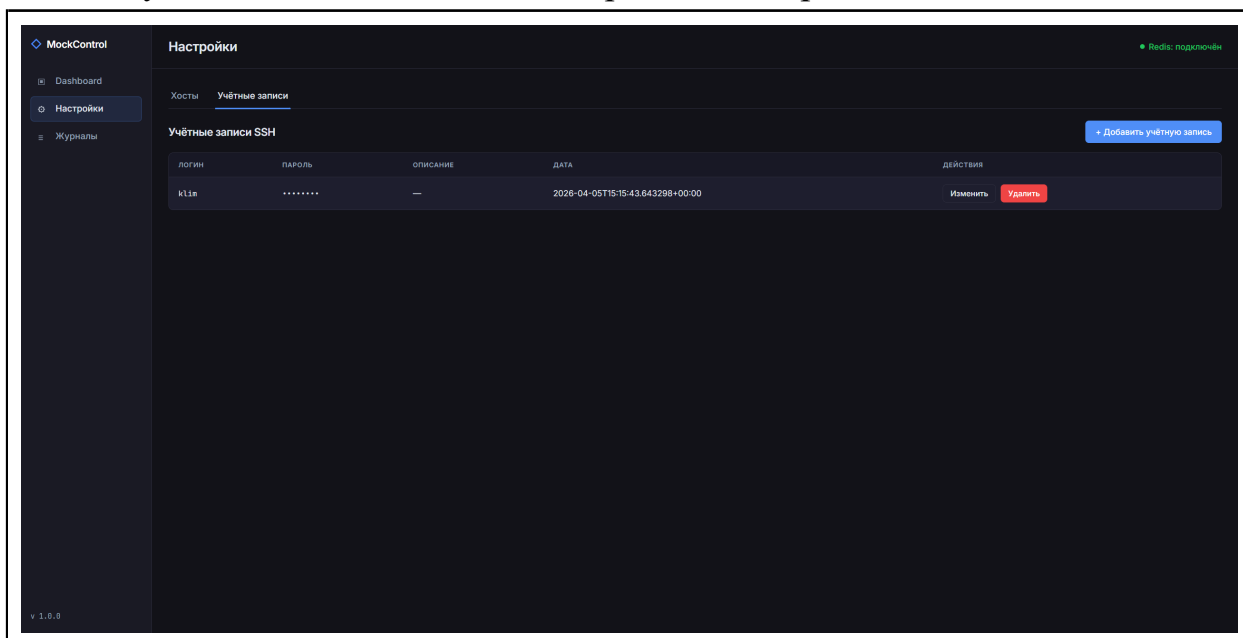


Рисунок В.4 – Реализованная страница настроек, вкладка «Учётные записи»

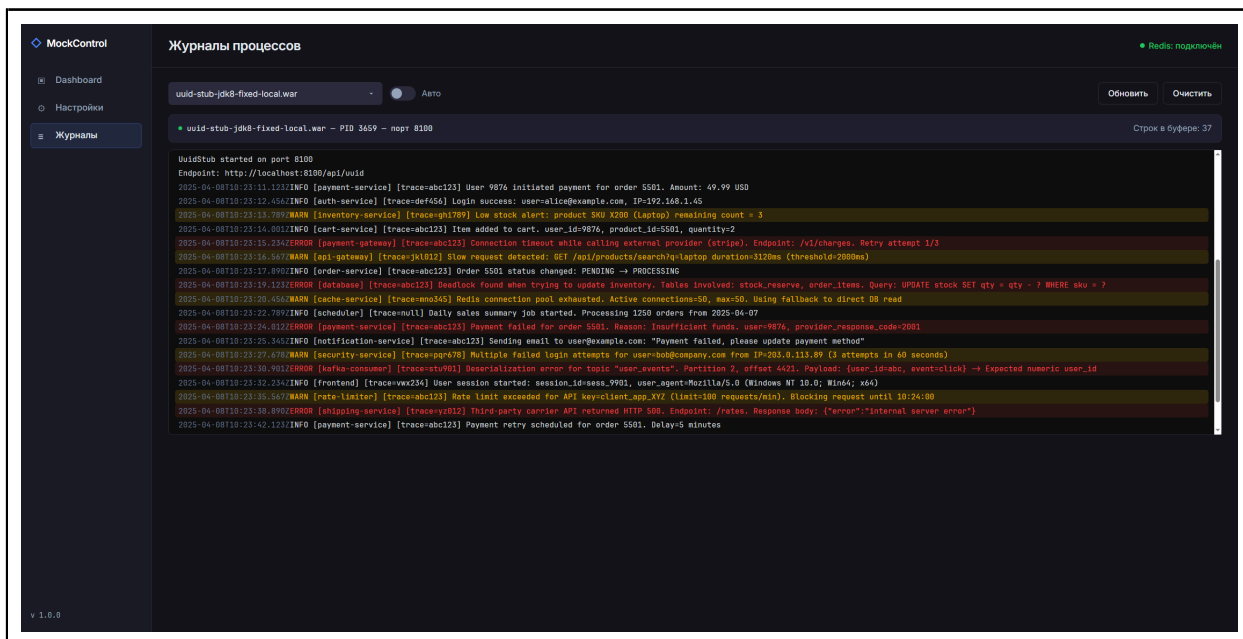


Рисунок В.5 – Реализованная страница просмотра журналов процессов